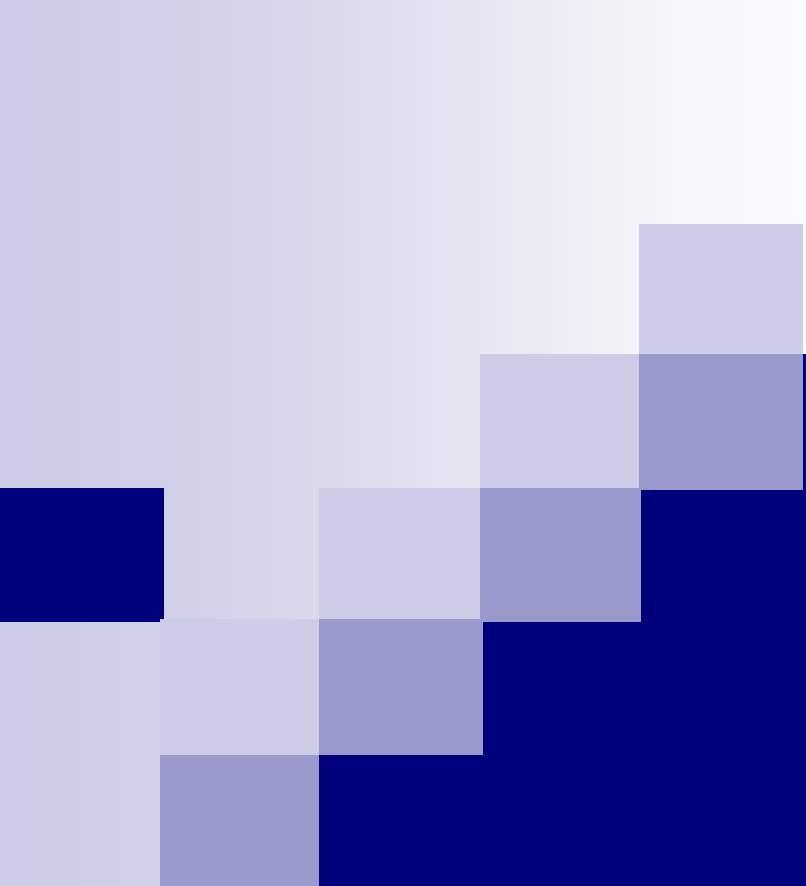
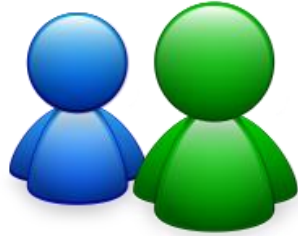




Patrones de diseño

Agenda



- Creacionales
 - Factory Method
 - Prototype
 - Singleton
 - Builder
- Estructurales
 - Adapter
 - Proxy
 - Facade
 - Composite
 - Decorator
- Comportamiento
 - Iterator
 - Mediador
 - Observer
 - State
 - Template method

¿Qué es un patrón?

- Effective
- Reusable



Pattern



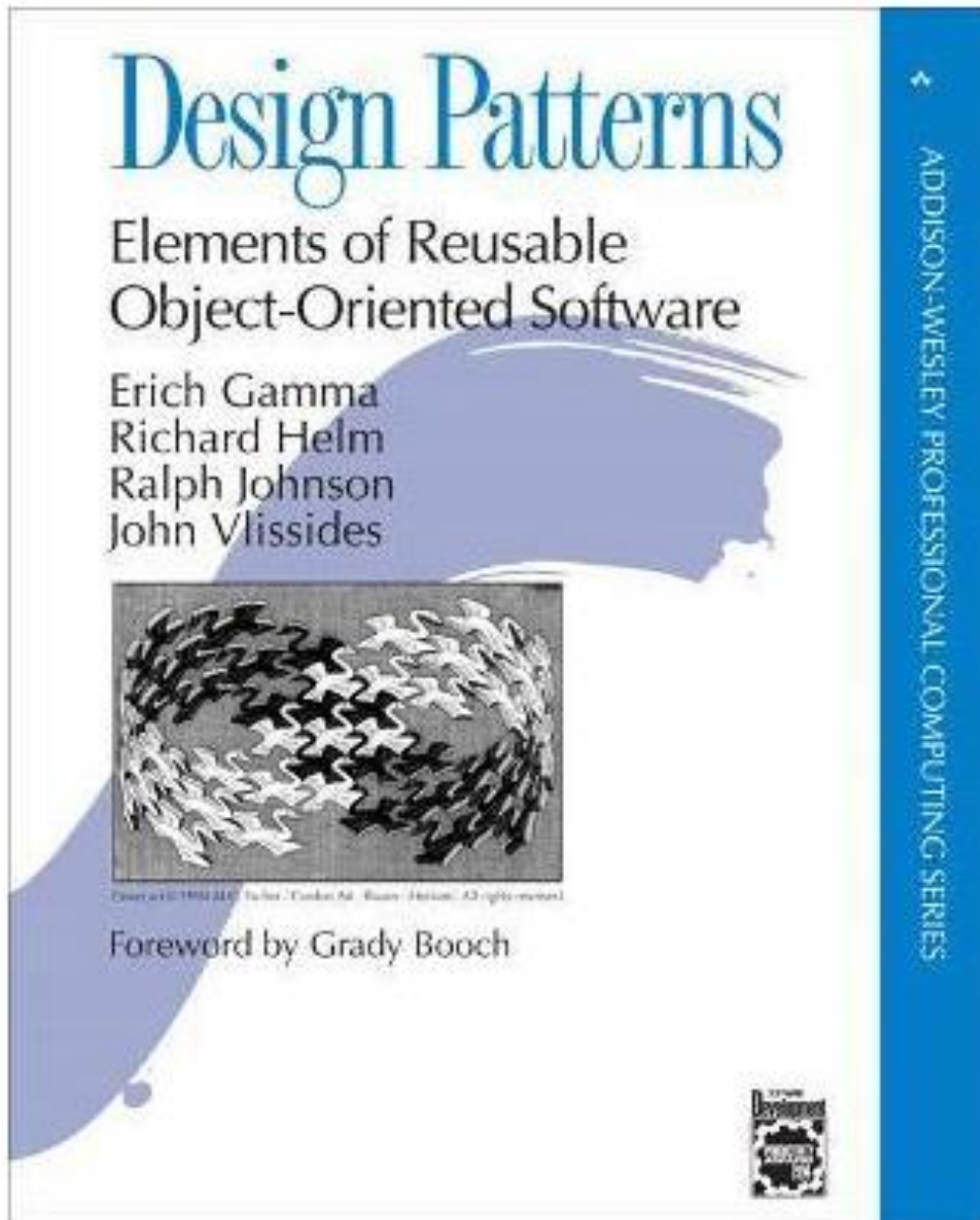
**Solución
estandarizada**



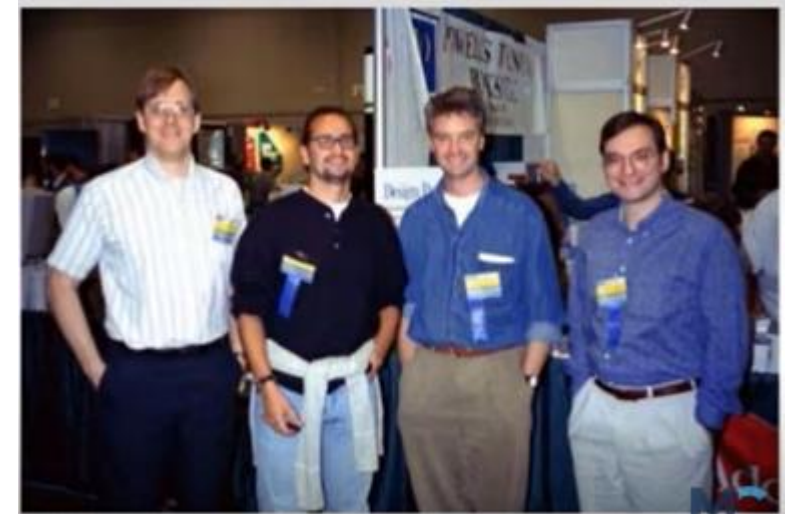
Abstracción



**Ventajas
Desventajas**



GoF



Gang of Four

Proponen soluciones sólidas a **problemas concretos** y probadas por la comunidad

Buscan **maximizar la cohesión** y **minimizar el acoplamiento**

Se basan en principios **SOLID** que favorece el código limpio
Evitan resolver un problema partiendo de cero



Beneficios

- Proporciona catálogo de elementos reusables en diseño de sistemas
- Evita la reiteración en la búsqueda de soluciones a problemas conocidos y solucionados anteriormente
- Formaliza un vocabulario común entre diseñadores
- Estandariza el diseño
- Facilita el aprendizaje a nuevas generaciones

¿QUÉ SON LOS PATRONES DE DISEÑO?

Son técnicas para resolver problemas recurrentes de diseño de software.



COMPORTAMENTALES

Establece soluciones relacionados con el comportamiento de la aplicación respecto a la interacción entre objetos y clases.

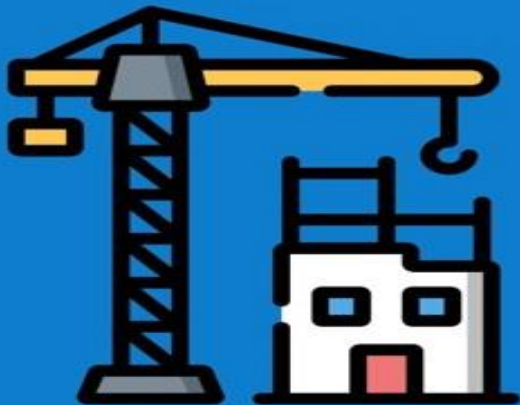


Ej. Observer, State.

Si tienes un problema de diseño es muy probable que ya exista un patrón que lo solucione, no reinventes la rueda.

CREACIONALES

Soluciona la creación de objetos, hace un sistema independiente de cómo sus objetos son creados.



Ej. Singleton, Factory.

ESTRUCTURALES

Describe cómo los objetos se componen para formar estructuras complejas.



Ej. Adapter, Decorator.

Patrones creacionales

Factory Method

Prototype

Singleton

Builder



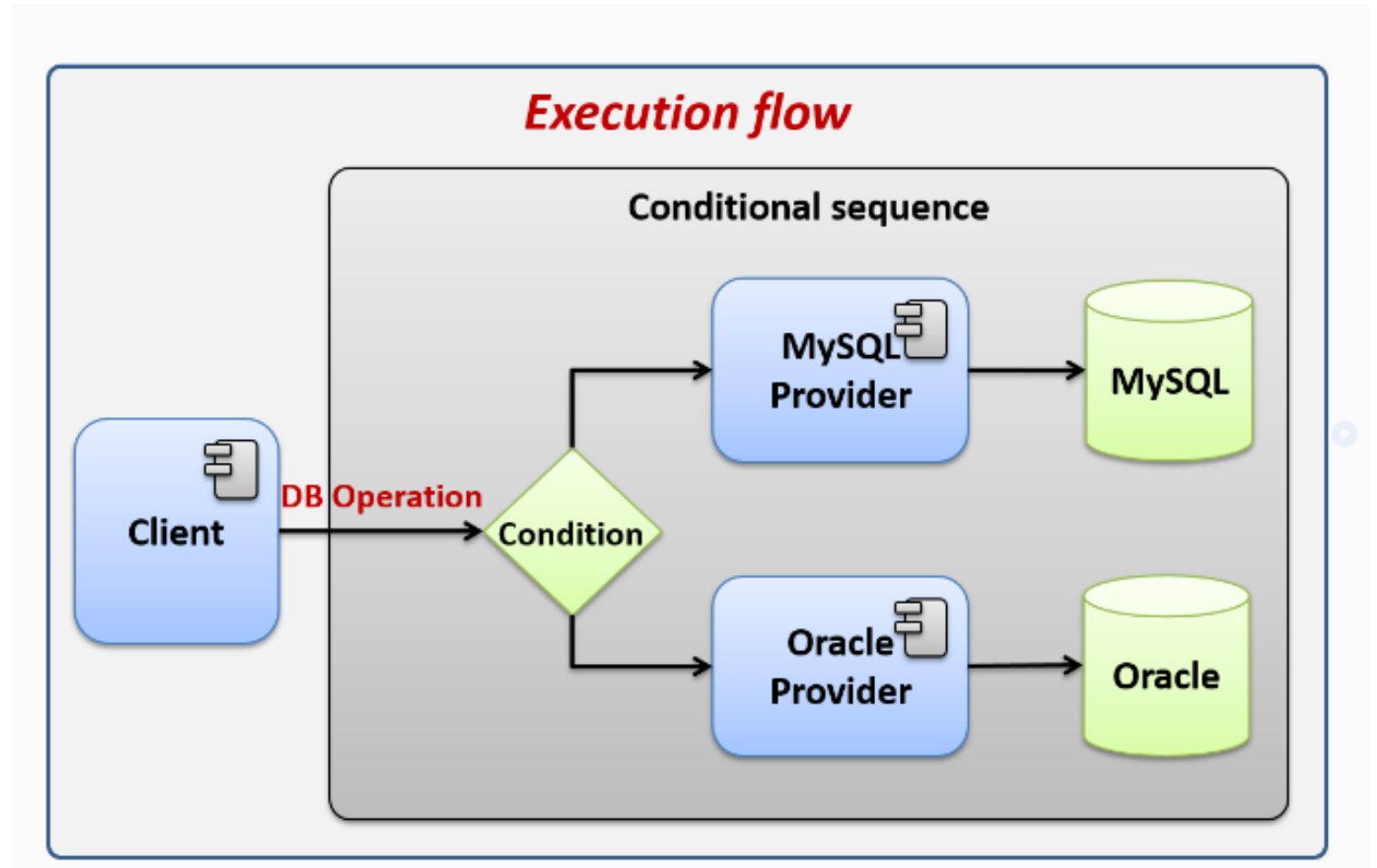
Factory Method

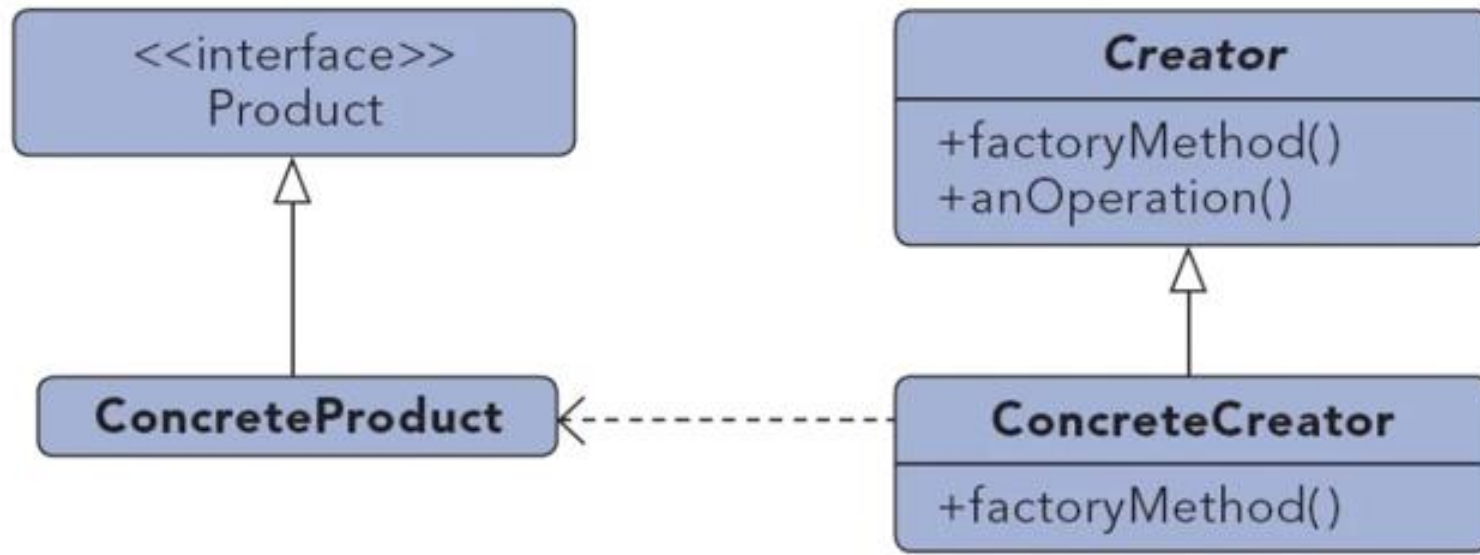
Define una interfaz para **crear objetos** en una superclase, pero deja en manos de la subclase la decisión de que clase concreta instanciar.

Define un constructor “virtual”

Factory Method

Crearemos una aplicación que sea **capaz de conectarse a más de una base de datos**, intercambiando de una a la otra, con tan solo realizar una simple configuración y sin una sola línea de código adicional.





Aplicación

Cuando una clase **no puede anticipar** el tipo de objeto que ésta debe crear
Las subclases son las que especifican los objetos que se crean

11

Utilidad

Encapsula la creación

En lugar de tener una clase central con diversos parámetros que varían en función de la autenticación del usuario, el proceso de autenticación puede delegarse en las clases Factory, que toman decisiones independientes sobre la gestión de cada usuario.

A close-up, high-contrast image of a mechanical or biological structure, possibly a robotic arm or a complex circuit. The word "[PROTOTYPE]" is overlaid in a bold, white, sans-serif font with a black outline, positioned horizontally across the middle of the image. The background is dark and textured, with some red and orange highlights suggesting heat or light. The image is partially obscured by a curved white border on the right side.

[PROTOTYPE]

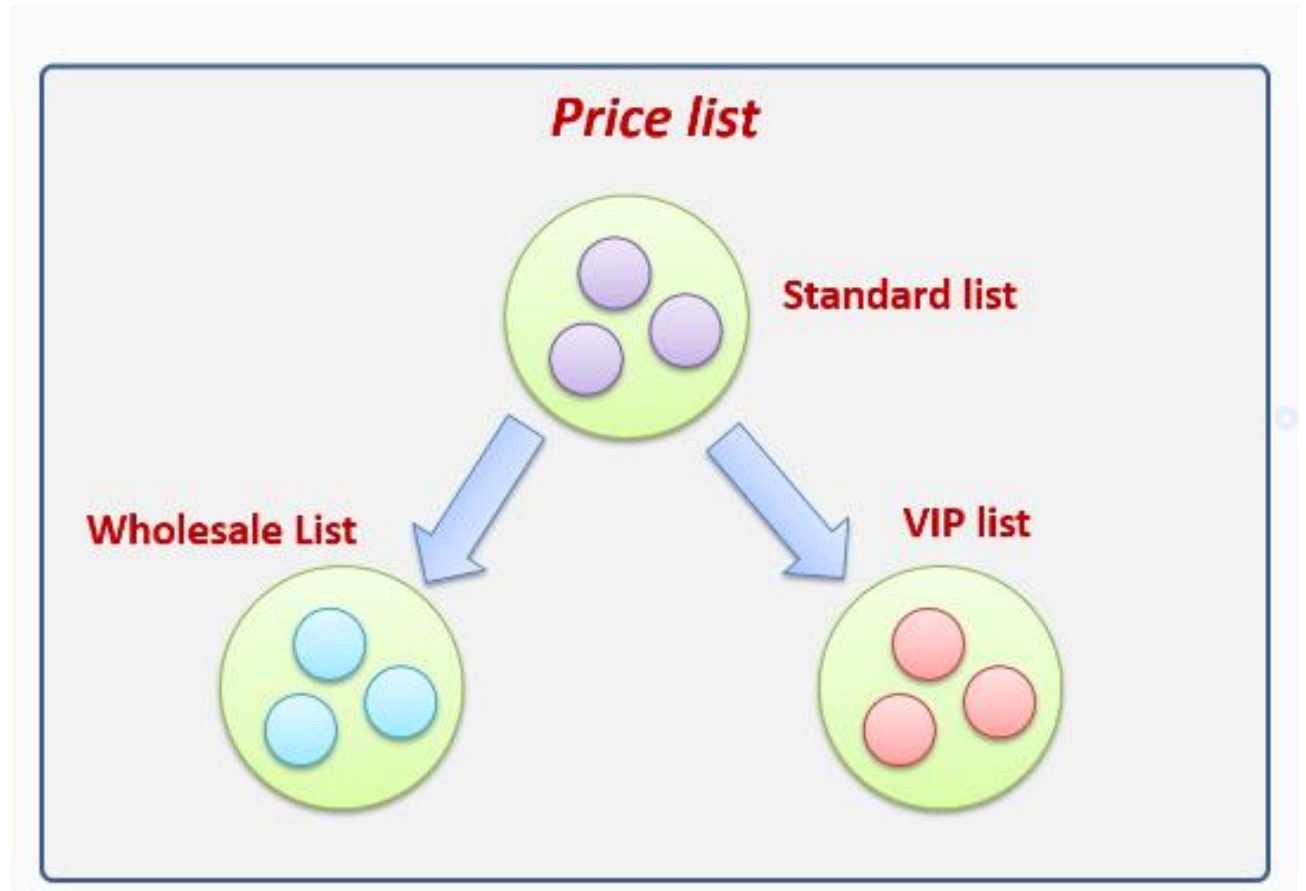
Prototype

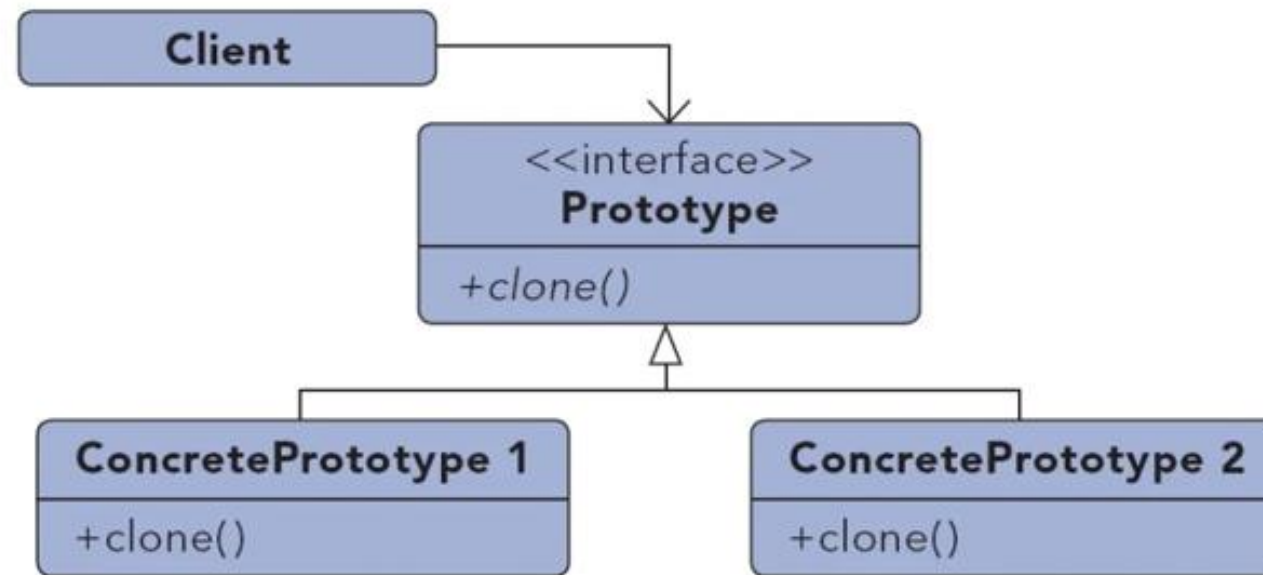
Tiene como finalidad crear nuevos objetos **clonando una instancia creada previamente.**

Prototype

Las empresas manejan más de una lista de precios, y estos listados tienen que ser creados desde cero con todos los productos del inventario.

En lugar de eso, utilizamos una lista de precios como base, y a partir de allí creamos listas derivadas





Aplicación

La composición, creación y representación de los objetos debe desacoplarse de un sistema
Las clases que se crean se especifican en tiempo de ejecución
Existe un número limitado de combinaciones de estado

16

Utilidad

Se requiere que objetos o estructuras de **objetos sean idénticos o se parezcan mucho**
Cada vez que en mi código necesite una instancia nueva. Tomo el creado y lo clono y trabajo con este. Obvio que queda igual, si quiero hacer una personalización lo puedo hacer.

SINGLETON

— DESIGN PATTERN SERIES

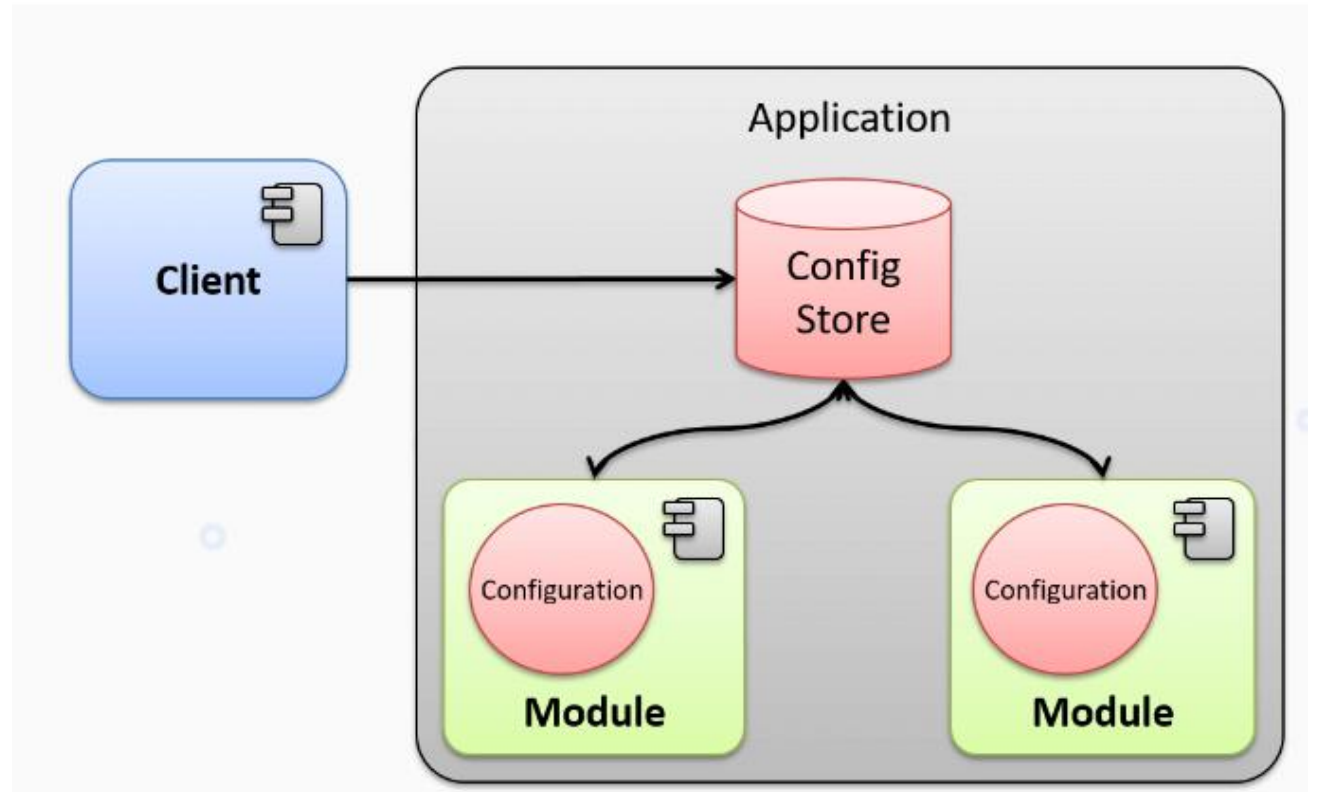
Singleton

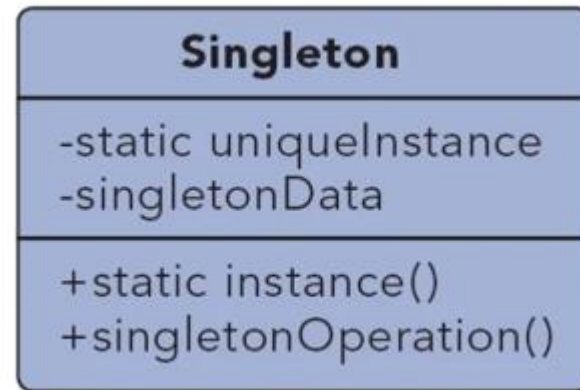
El objetivo es garantizar que una clase sólo tenga **una instancia** y proporcionar un punto de acceso global a ella

Singleton

Gestionar la configuración del sistema desde un único punto centralizado.

Así, cuando la aplicación comienza, cargará la configuración inicial y está disponible para toda la aplicación.





Aplicación

Cuando se necesites **un control más estricto de las variables globales**.
Ej. Un gestor de ventanas, una única cola de impresión.

20

Utilidad

El uso común de este patrón se encuentra en componentes que quieren restringir su uso desde un sólo punto.



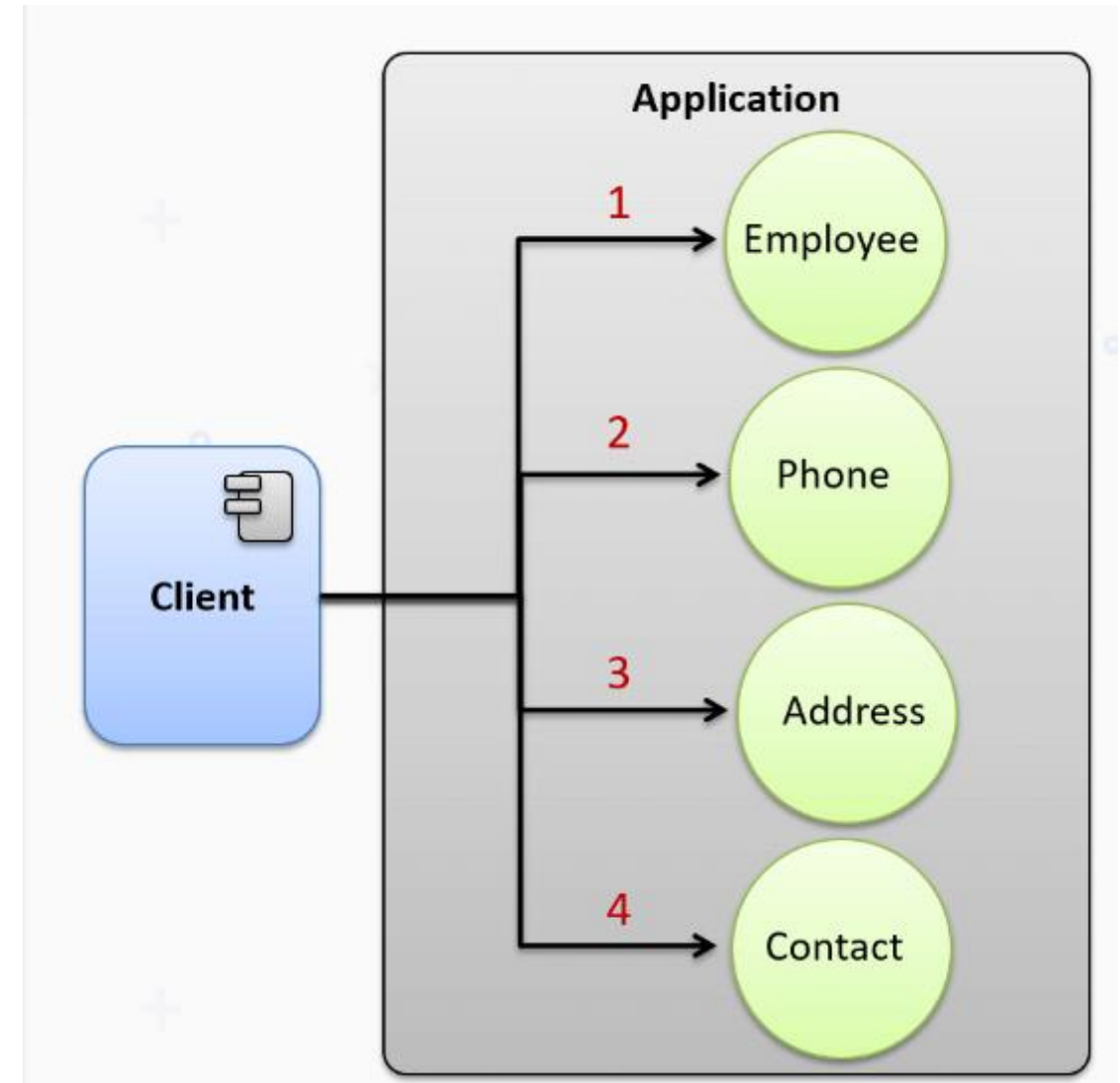
Builder

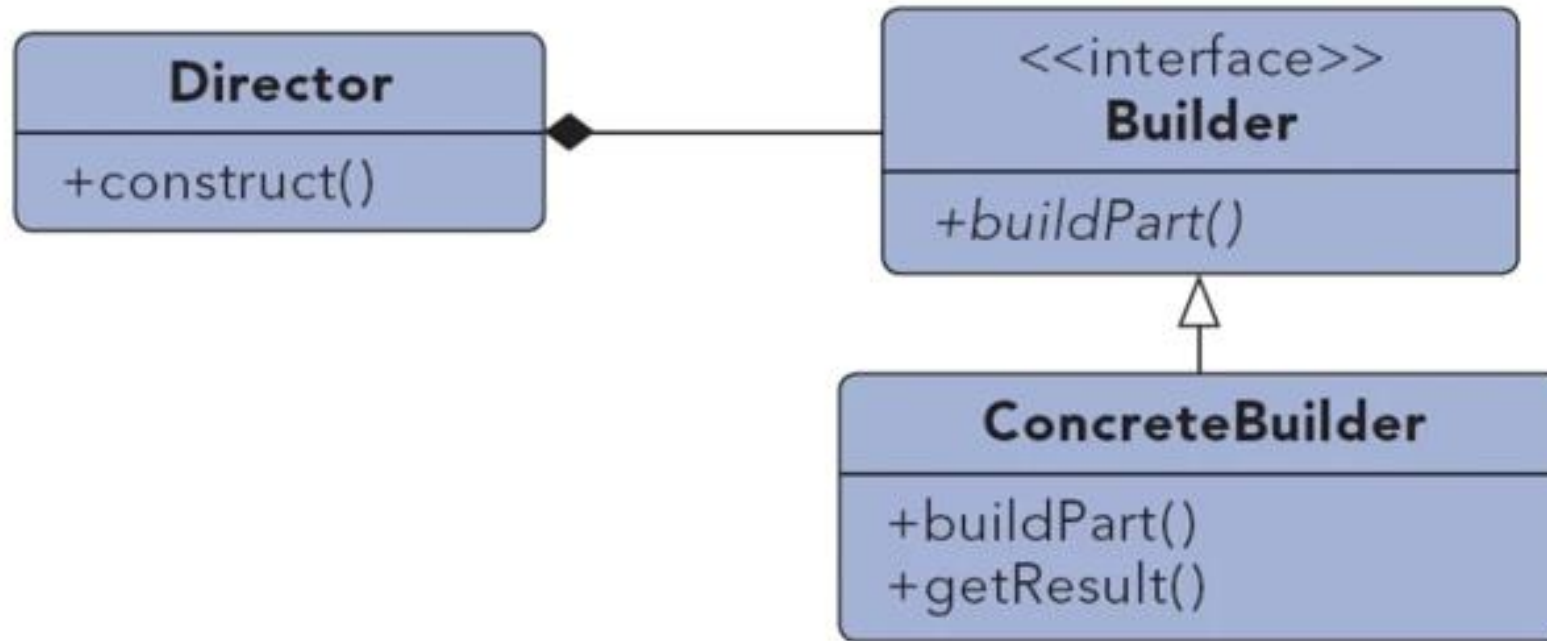
Centraliza el proceso de **creación de un objeto en un único punto**, de tal forma que el mismo proceso de construcción pueda crear representaciones diferentes

Builder

Se utiliza en situaciones en las que el objeto tiene una gran cantidad de atributos en el constructor.

Con Builder, **nos permitirá crear la estructura completa por** ej. de un empleado con todas sus datos, como lo son: su dirección, teléfonos, contactos, etc.





Aplicación

Reduce acoplamiento

Se **independiza del código de construcción**, (responsabilidad única)

25

Utilidad

Reutilizar código de construcción al construir varias representaciones de productos

Se requiere control sobre el proceso de creación en tiempo de ejecución

Patrones Estructurales

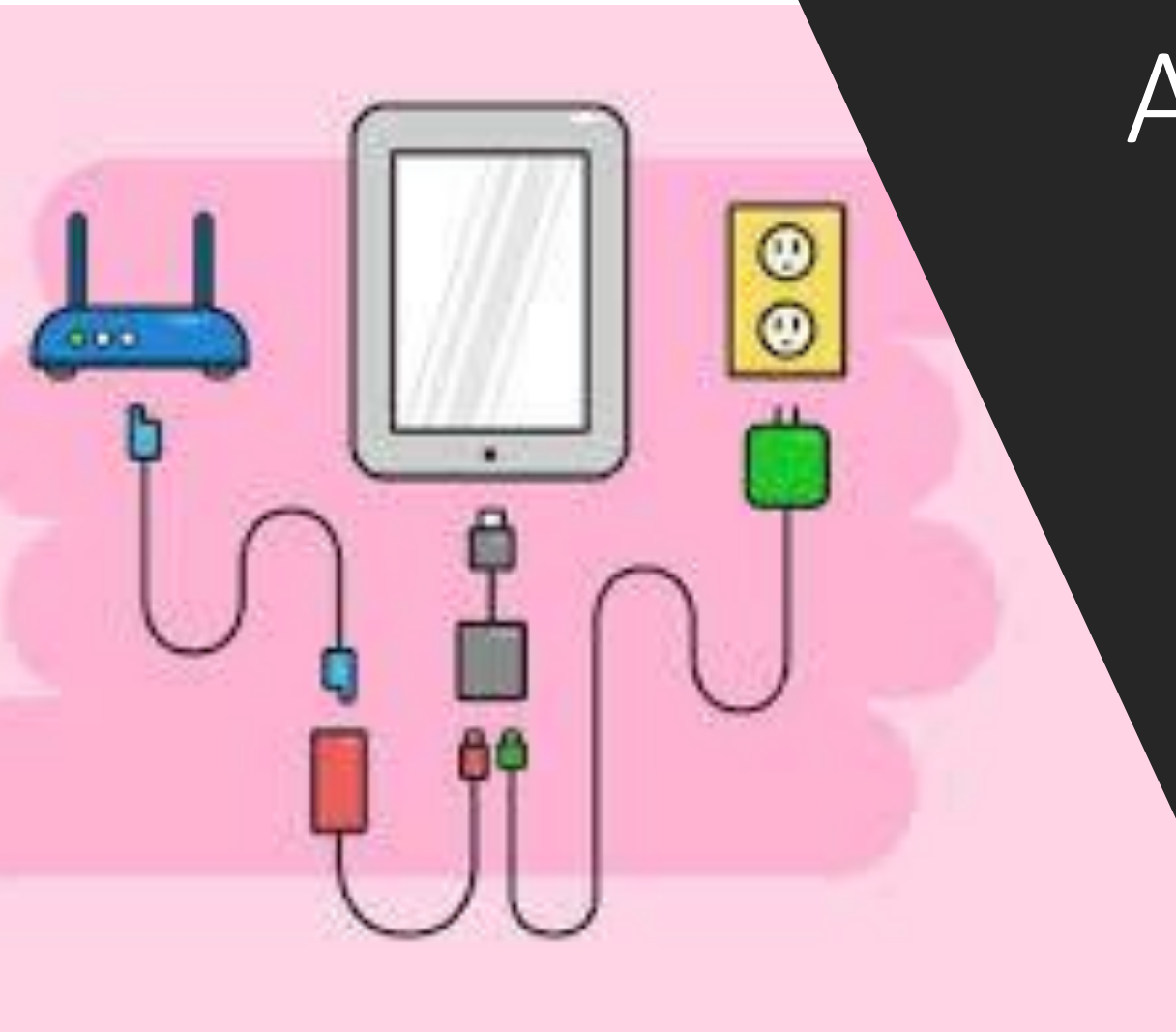
Adapter

Proxy

Facade

Composite

Decorator



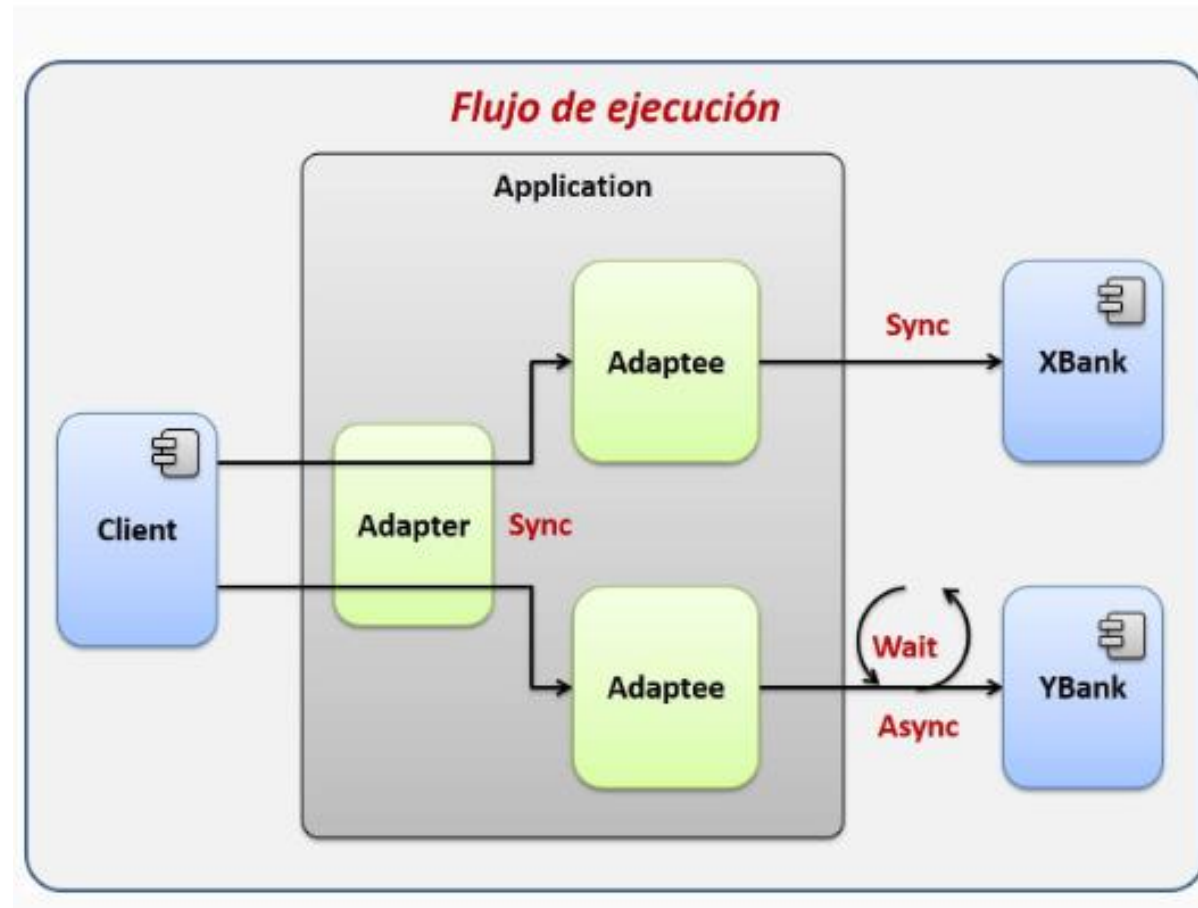
Adapter

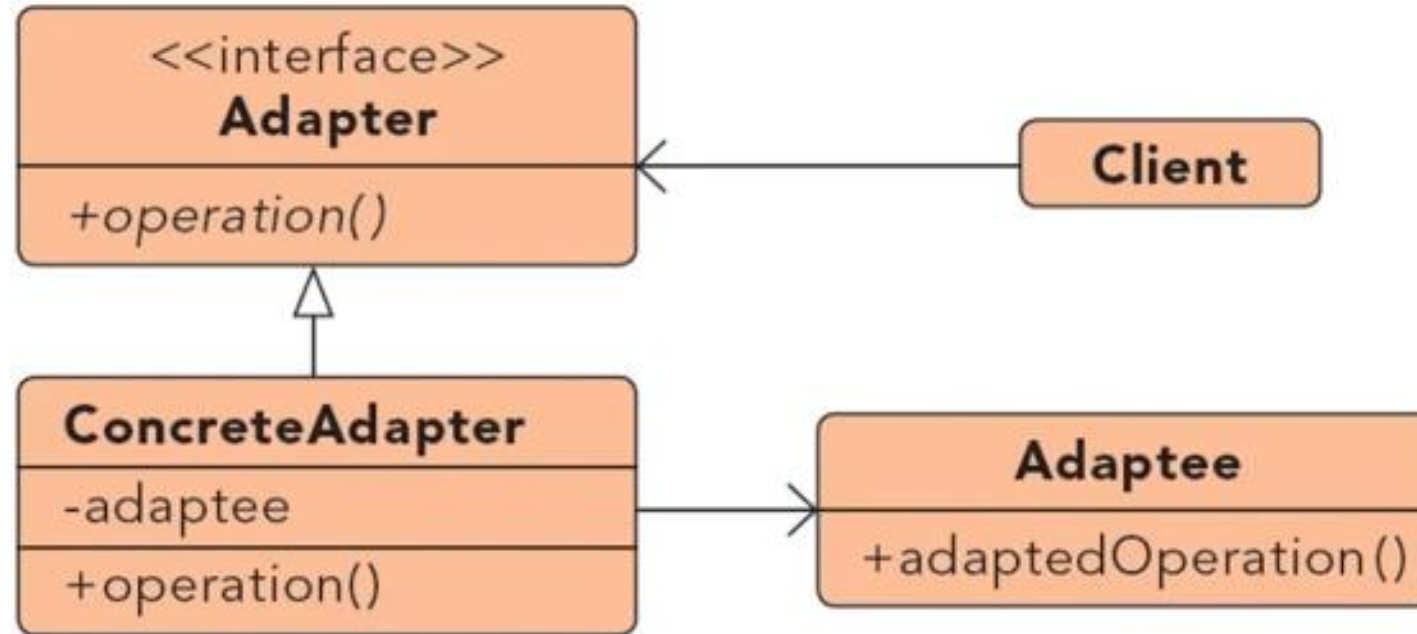
Actúa como un conector **entre dos interfaces que son incompatibles** y que no se las puede conectar directamente

Adapter

Contamos con dos APIs con interfaces diferentes y (aunque su funcionamiento es prácticamente igual), lo que implica tener dos implementaciones para procesar los préstamos con cada banco

Mediante este patrón **crearemos un adaptador que permitirá ocultar la complejidad de cada implementación, exponiendo una única interface compatible**, además que dejáramos el camino preparado por si el día de mañana llegara una nueva API bancaria





Aplicación

Cuando trabajamos con **librerías externas y frameworks**

35

Utilidad

Integración de componentes de terceros
Soporte a código legacy



Proxy

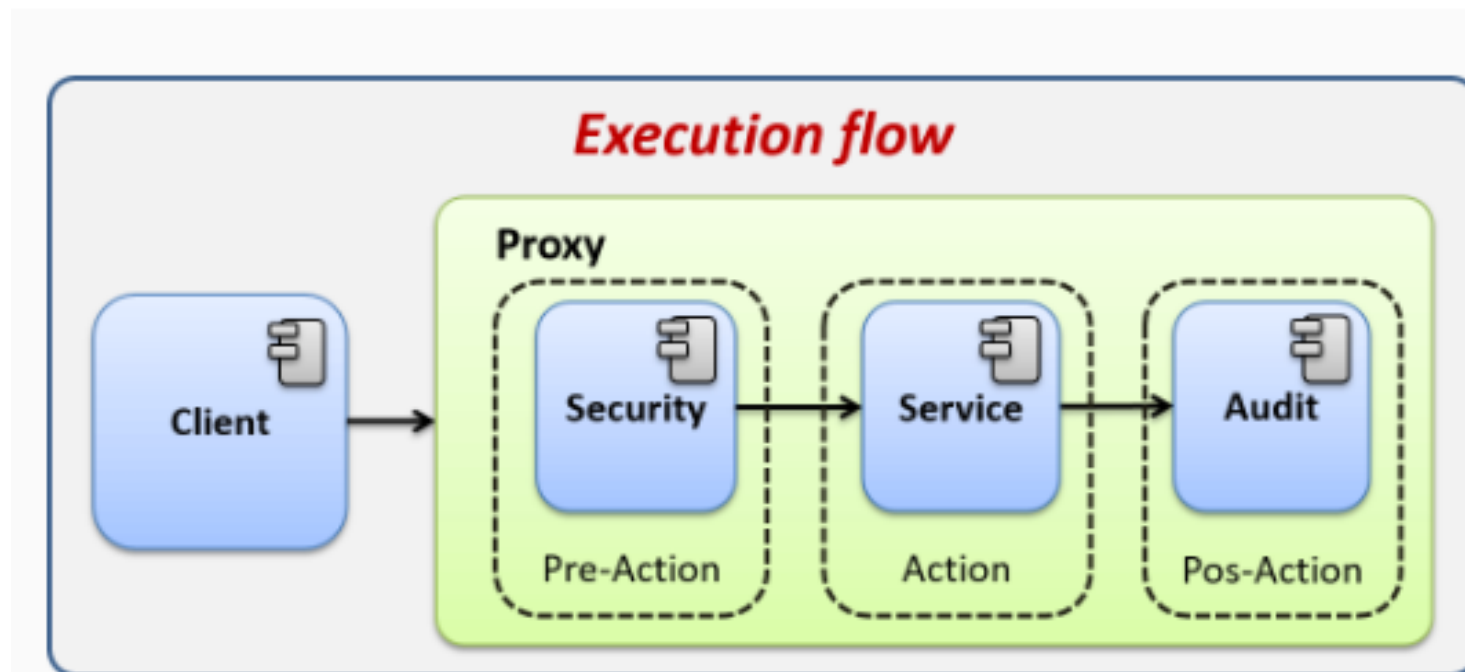
Una clase que permite
operar con un objeto de
otra clase exponiendo una o
más de sus interfaces

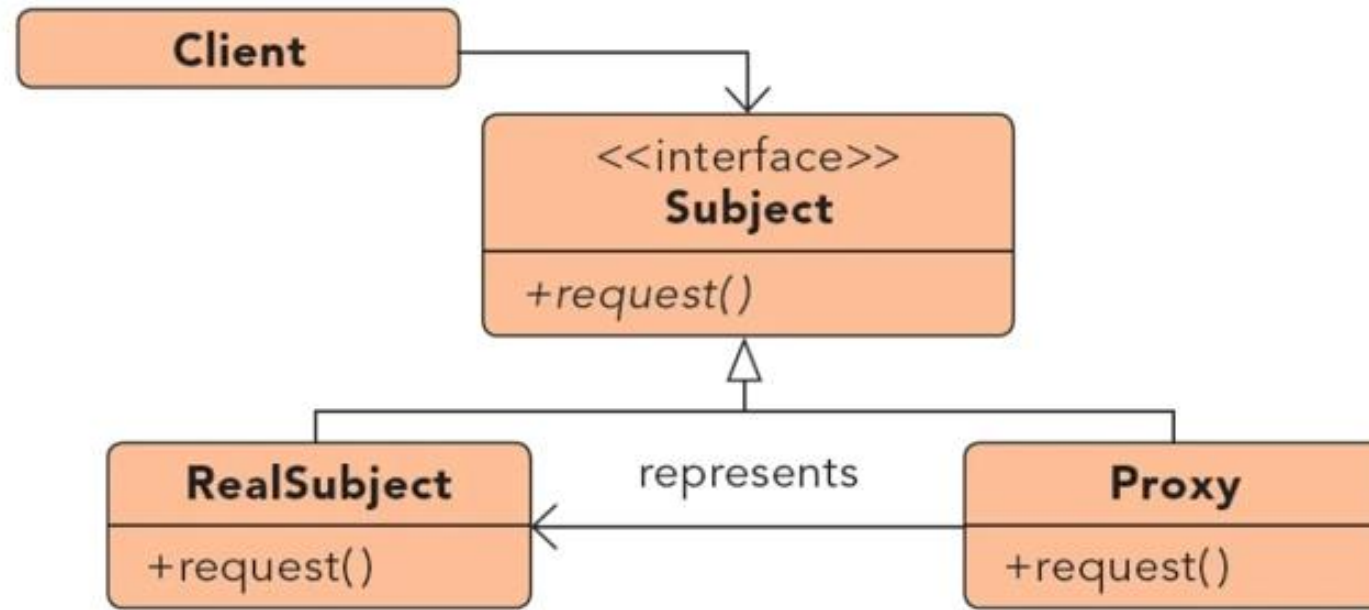
Proxy

Crearemos un mecanismo de seguridad, el cual intercepte las ejecuciones de procesos para validar que el usuario tenga los privilegios necesarios.

Una vez que el proceso es ejecutado, se auditará la ejecución y quedará un registro de la ejecución.

Todo esto se realizará sin que el usuario se de cuenta, pues el proxy **envolverá la lógica de seguridad**.





Aplicación

Provee seguridad que como el proxy es el único que interactúa con el objeto, el estado del mismo solo se modifica a través del proxy
Gestiona objetos que consumen muchos recursos, Evita que haya muchas instancias de los mismos

Utilidad

El proxy funciona como un filtro hay cosas que las deja pasar y otras no.
Proxy implementa las interfase y conoce el servicio al que quiero mandar y establece controles para dejar que pasen los objetos.



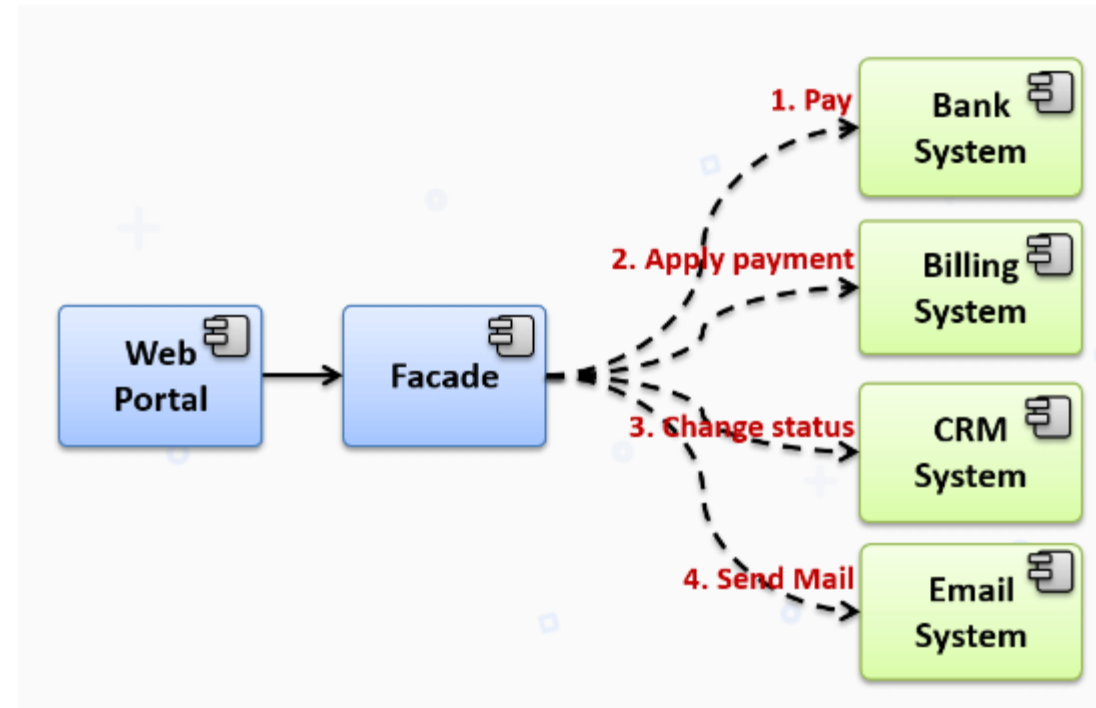
Facade

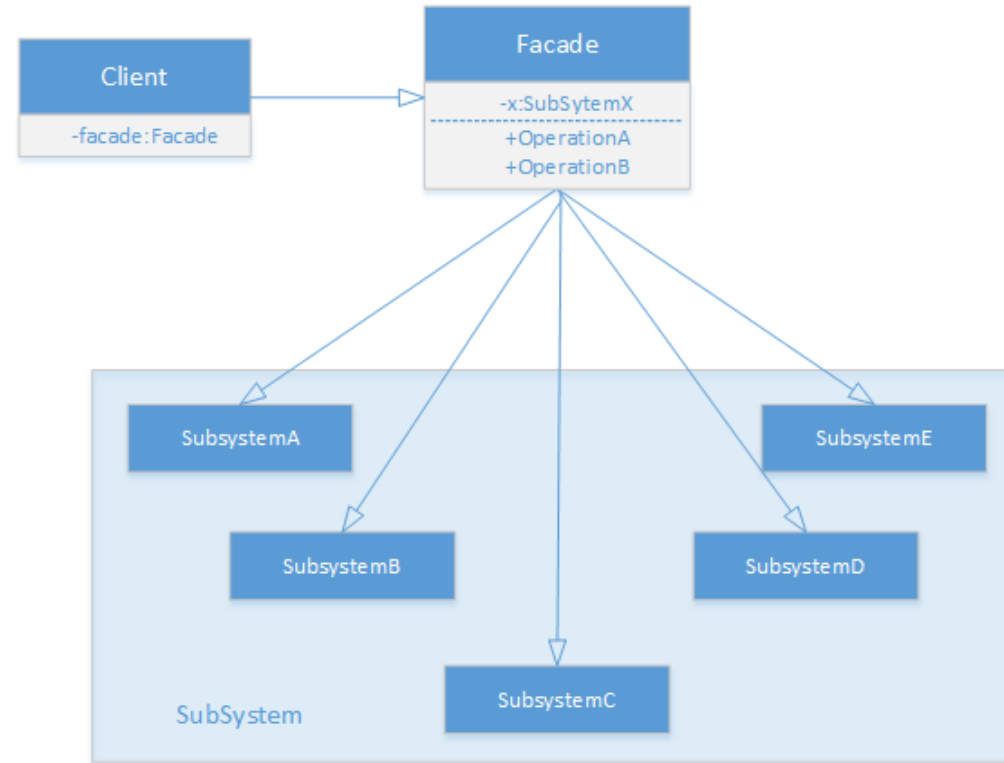
Proporciona una **interfaz unificada** para un conjunto de interfaces de un subsistema

Facade

Implementaremos un sistema que permite realizar pagos en línea, para lo cual será necesario interactuar con varios sistemas.

Para ello se implementará una fachada que **permite exponer operaciones de alto nivel**, (las que se encargan de interactuar con los subsistemas) y abstrae a los programadores de la complejidad de interactuar con dichos sistemas.





Aplicación

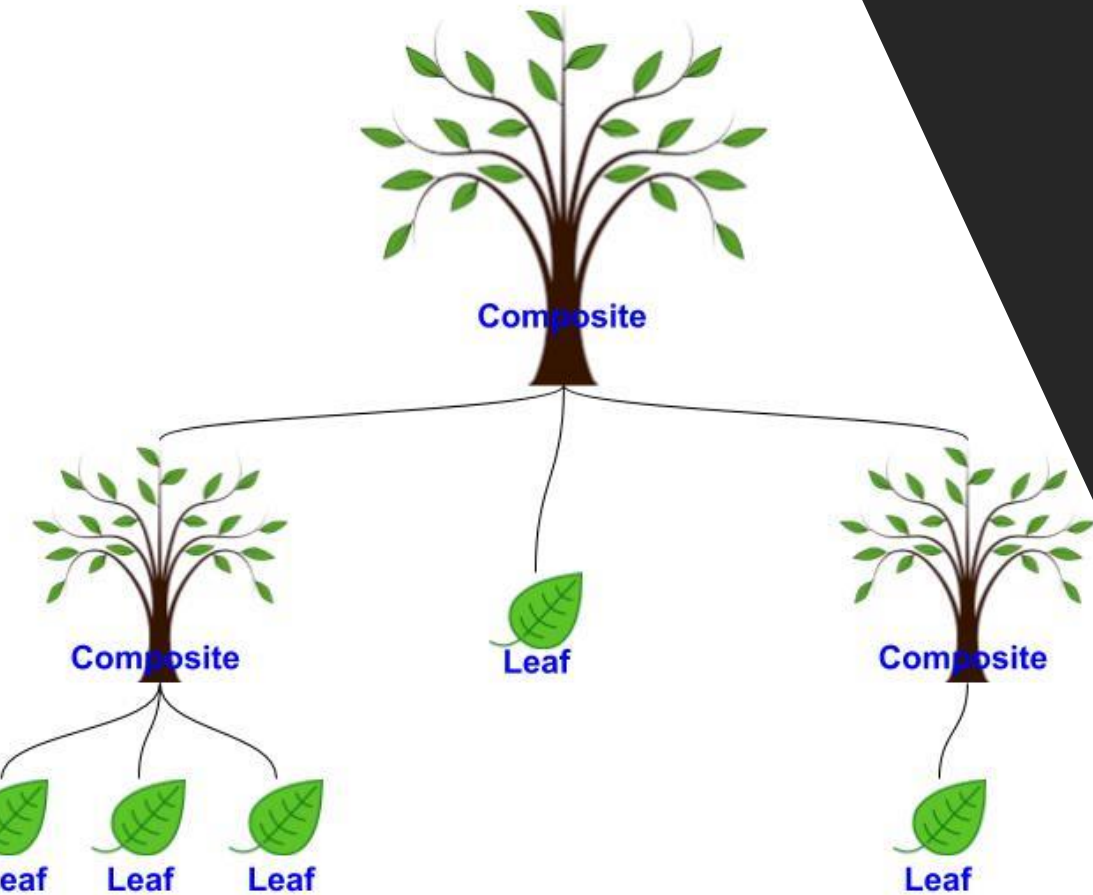
Implementa una **interfaz universal** para las distintas interfaces del subsistema o subsistemas. Y (si es necesario) puede realizar **funciones adicionales** antes o después de reenviar una solicitud al cliente.

Utilidad

Simplifica en el caso que haya muchas dependencias entre las implementaciones de sistemas y los clientes

Composite

Se aplica para construir **objetos complejos a partir de otros más simples (y similares entre sí)**, gracias a la composición recursiva y a una estructura en forma de árbol.

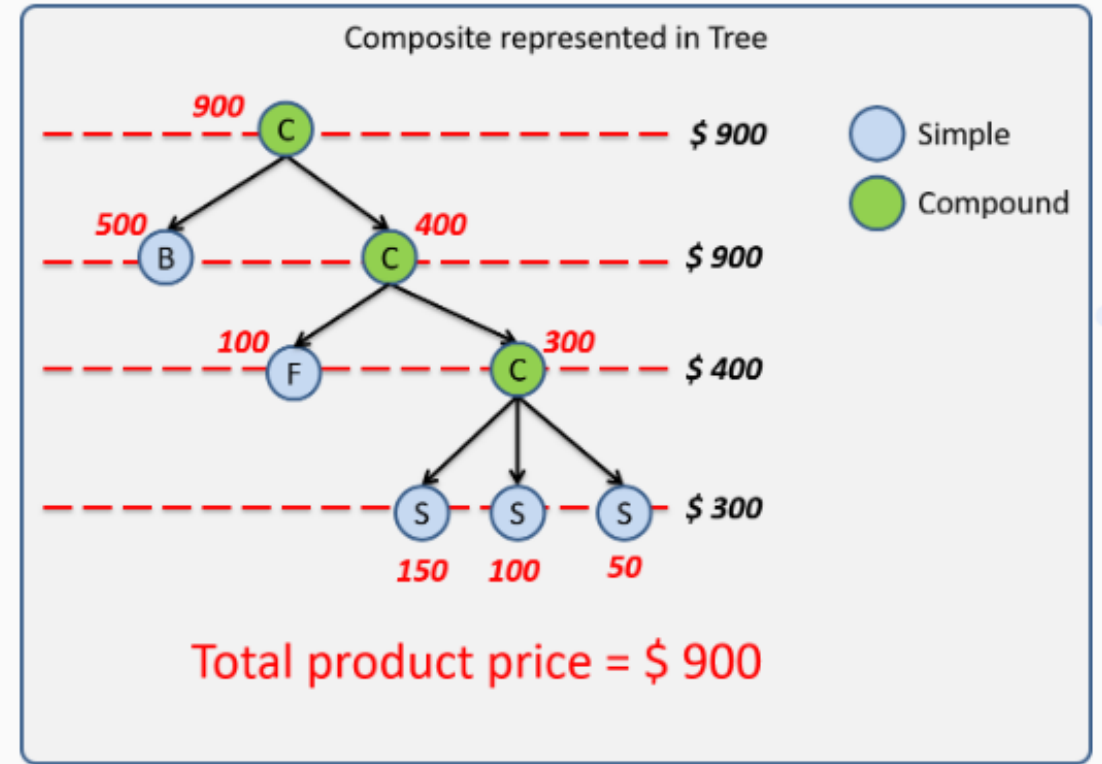


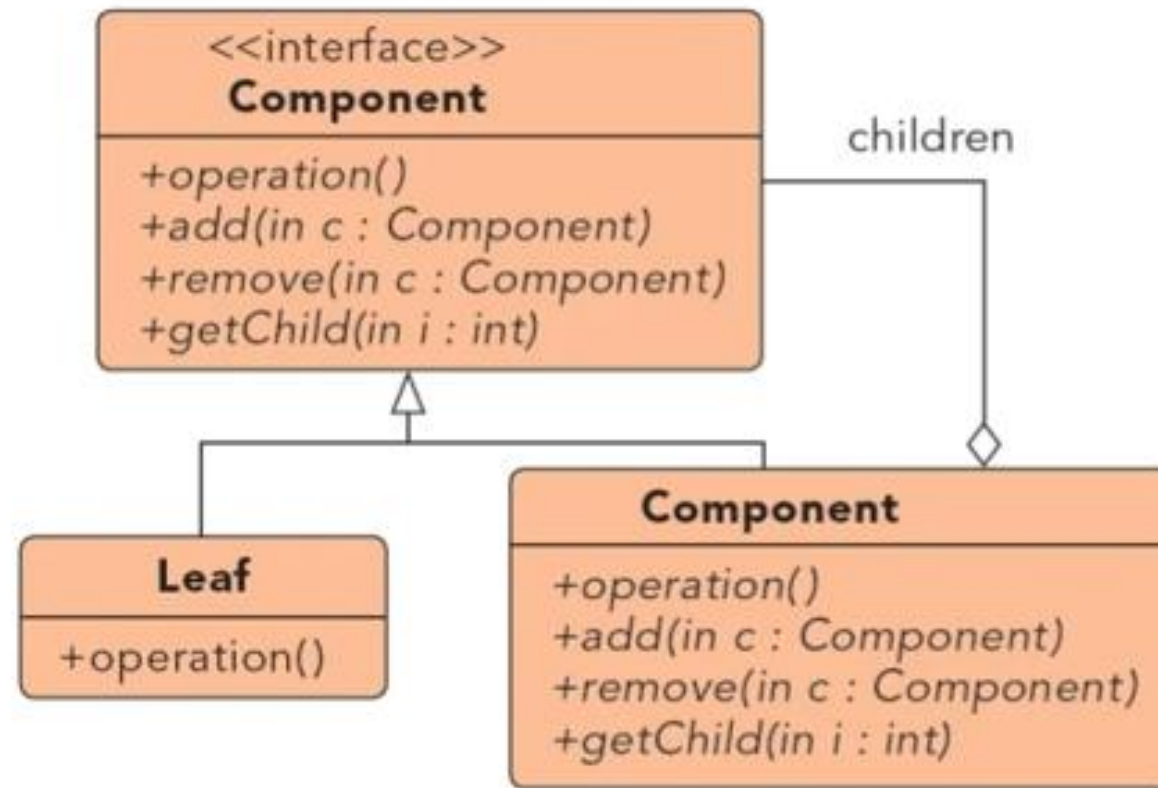
Composite

Implementamos una aplicación que nos permita crear una **estructura compleja de productos**.

Donde un producto puede **ser tan solo uno, o el agrupamiento** de más de un producto que puede ser representado como uno solo.

Las **agrupaciones de productos o productos compuestos**, pueden ser creados a su vez de otros productos compuestos, formando una estructura de árbol, donde el precio del producto es igual a la suma de todos los nodos de él, y son vistos desde el sistema, como producto único.





Aplicación

Facilita la **representación de estructuras altamente anidadas**
Código simple y conciso
Gran escalabilidad

44

Utilidad

Mejorar la gestión de problemas de diseño recurrentes en la programación orientada a objeto



Decorator

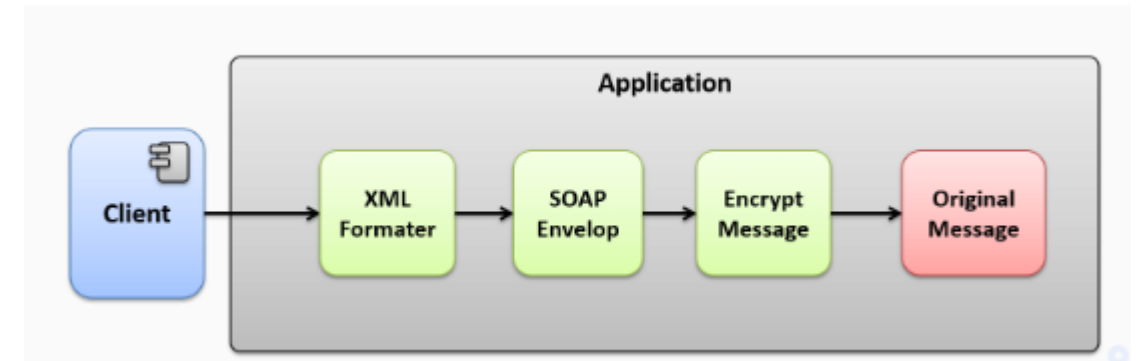
Añade **nuevas funcionalidades** a un **objeto** en tiempo de ejecución sin **modificar su estructura** y a través de una envoltura “wrapper”.

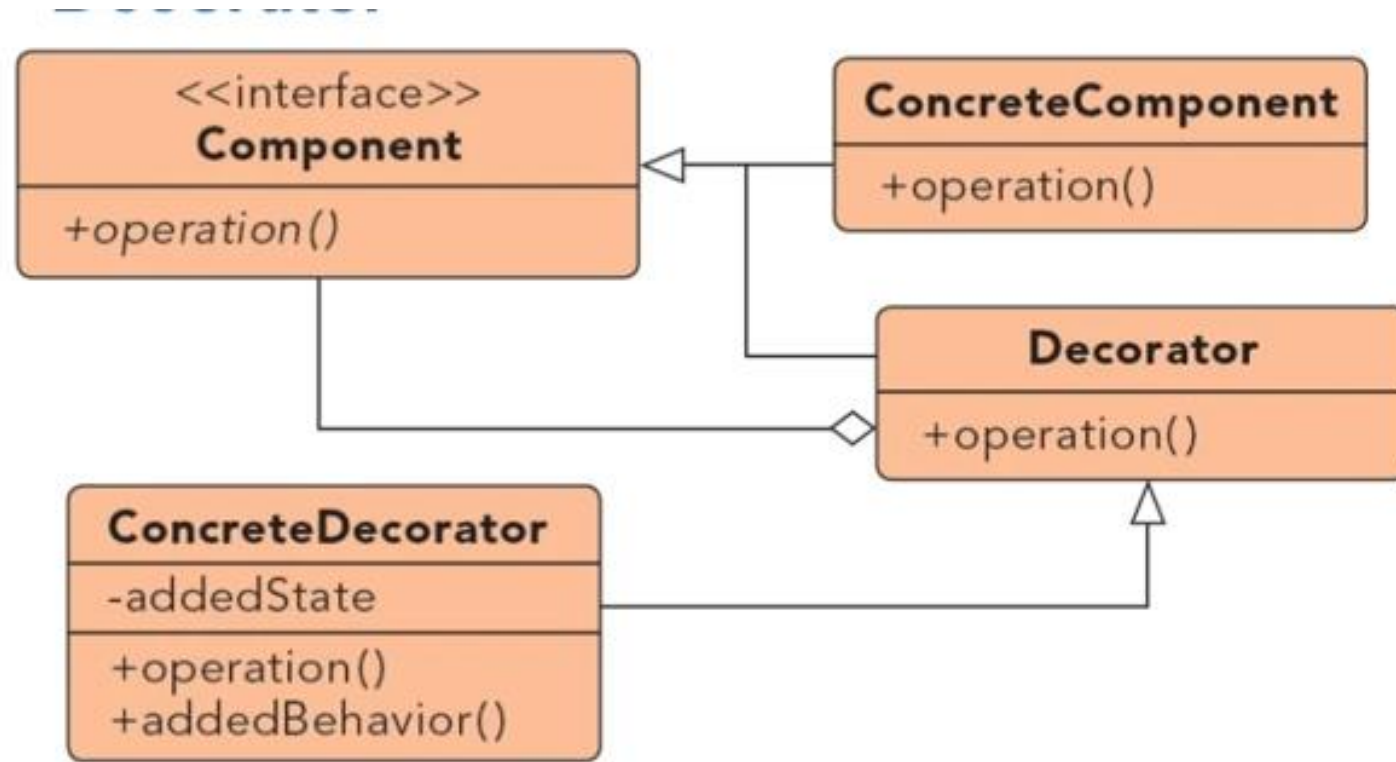
Decorator

Necesitamos procesar un mensaje en capas, **donde cada capa se encargará de procesar un mensaje a diferente nivel.**

- primero convertiremos un Objeto en XML,
- Luego lo envolveremos en un mensaje SOAP para después encriptar el mensaje,
- finalmente obtendremos un mensaje SOAP totalmente encriptado, el cual podrá ser enviado de forma segura a un destinatario.

Cada capa de procesamiento será implementada con un decorador, y cada decorador podrá cambiar de posición para obtener un resultado diferente, de la misma manera, se podrá agregar nuevos decoradores en medio de cualquier paso.





Aplicación

Proporciona una alternativa flexible a la herencia para extender la funcionalidad

47

Utilidad

Componentes del *software* orientado a objetos sean más flexibles y fáciles de reutilizar. Por ejemplo se desea añadir responsabilidades individuales a los objetos dinámicamente y transparentemente sin afectar a otros objetos

Patrones comportamentales

Iterator

Mediador

Memento

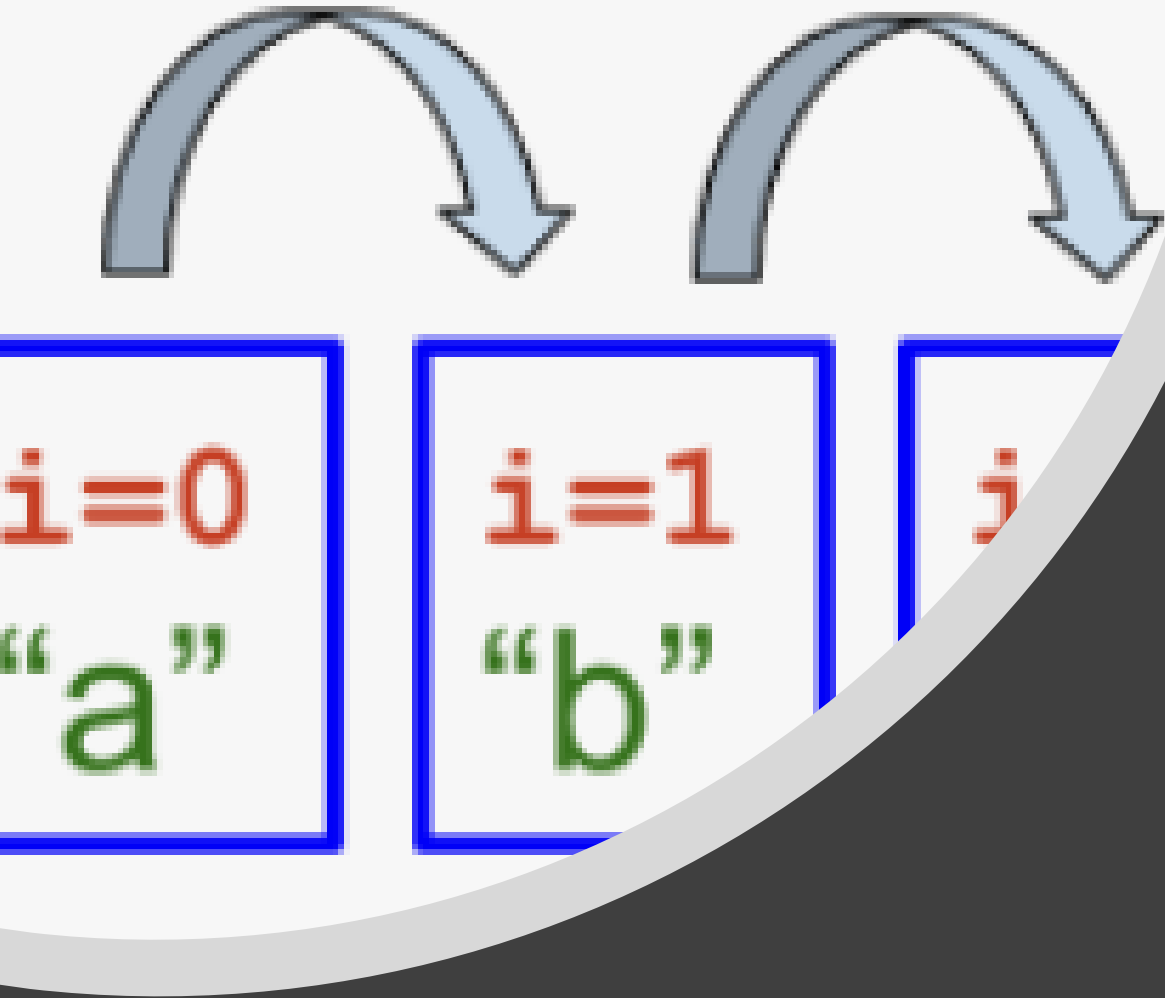
Observer

State

Strategy

Template method

Visitor



Iterator

Define una interfaz que declara los métodos necesarios para acceder de forma secuencial a una colección de objetos, como string o listas

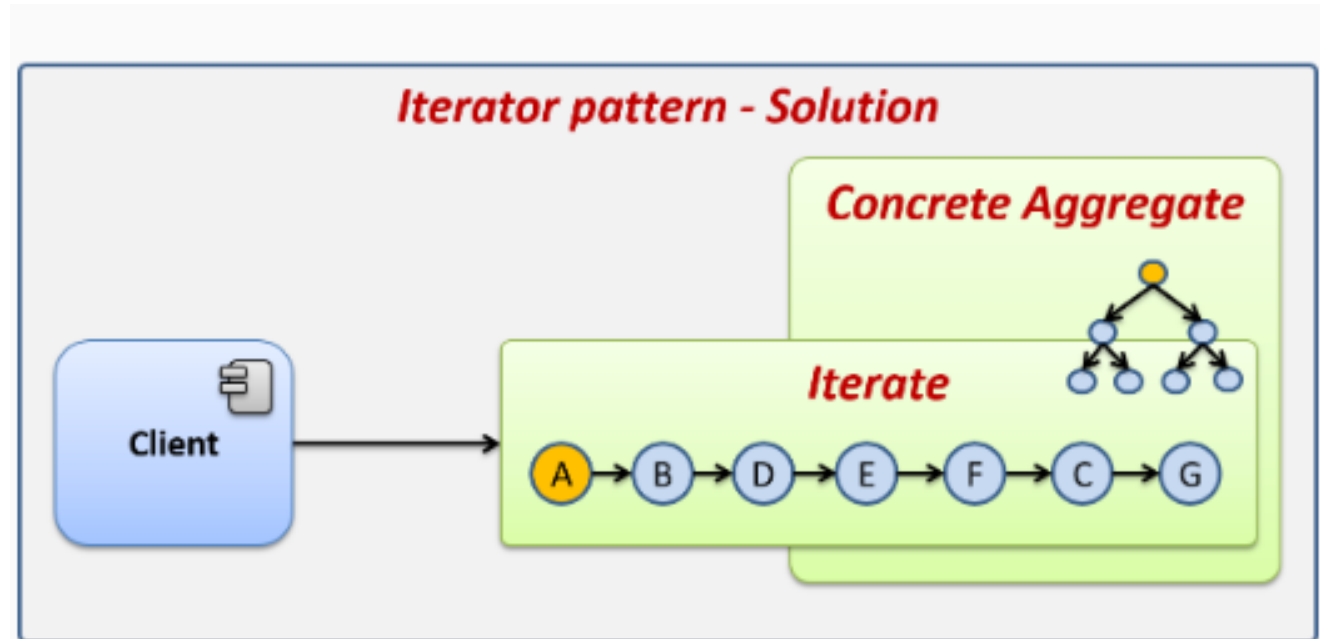
Sin tener que exponer su representación interna

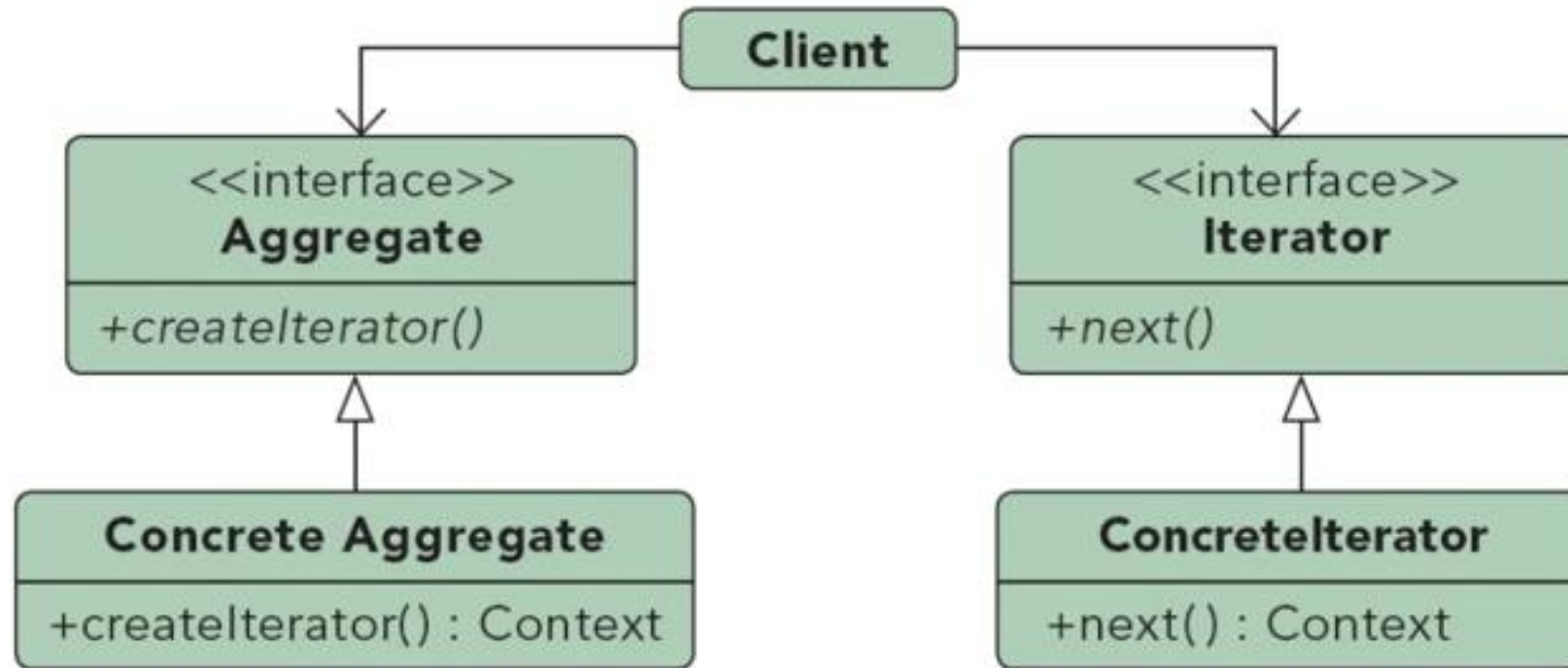
También conocido como "Cursor"

Iterator

Crearemos una aplicación que nos permita recorrer una estructura organizacional jerárquica, mediante la implementación de un iterador.

El cual nos permitirá recorrer todo el árbol de la estructura de forma secuencial.





Aplicación

Permite recorrer los elementos del agregado independientemente de su organización. Si todos los objetos agregado (listas, árboles, etc) generan un objeto Iterador con el mismo interfaz resulta muy fácil operar con ellos y se facilitan los cambios de implementación.

Utilidad

Se necesita tener acceso a elementos sin tener acceso a toda la representación
Se pretende hacer **recorridos múltiples o concurrentes a los** elementos

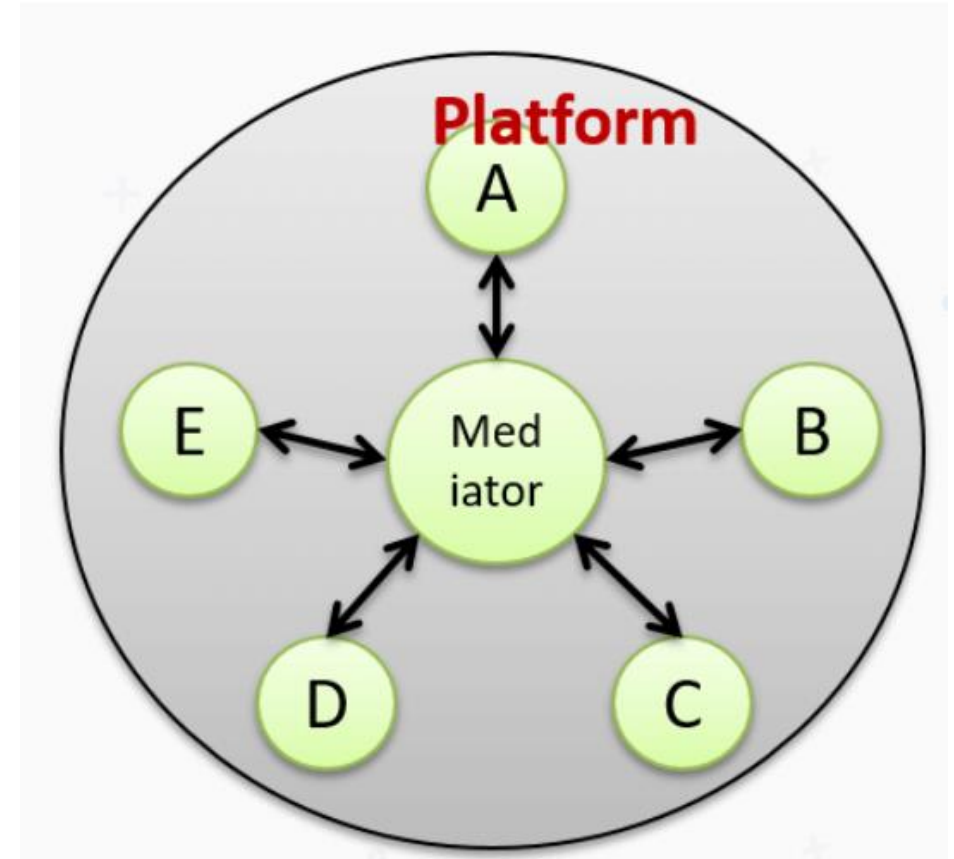


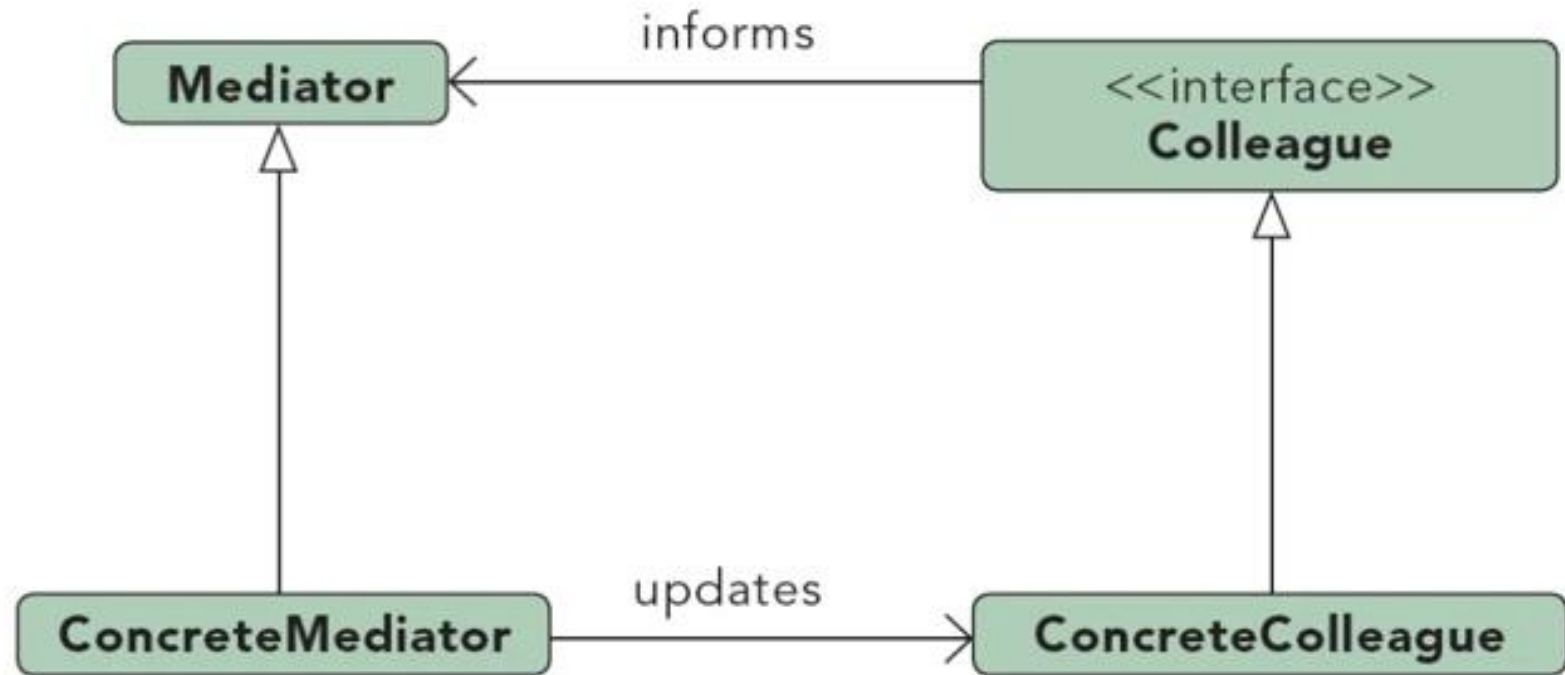
Mediador

Define un **objeto que encapsula** la manera en que interactúan un conjunto de objetos entre sí

Mediador

El Mediador, que funcionará como **punto central de comunicación** entre todos los módulos, eliminando la dependencia directa que existen entre los módulos para el funcionamiento.





Aplicación

permite **reducir las dependencias caóticas entre objetos**

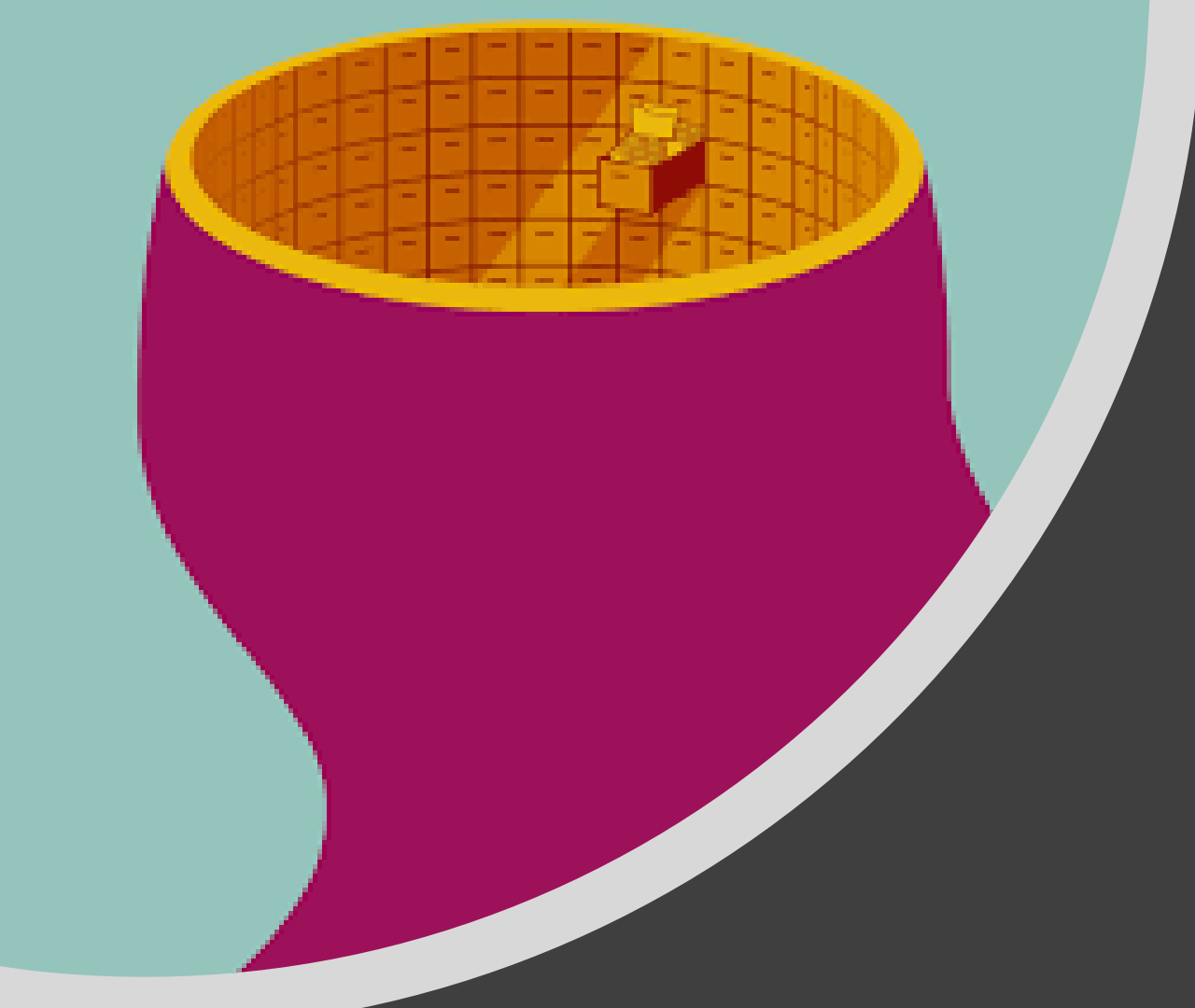
Simplifica la comunicación entre objetos

Centraliza el control, el mediator se encarga de la comunicación entre los objetos

54

Utilidad

Promueve un bajo acoplamiento al evitar que los objetos se refieran unos a otros explícitamente, y permite variar la interacción entre ellos de forma independiente.



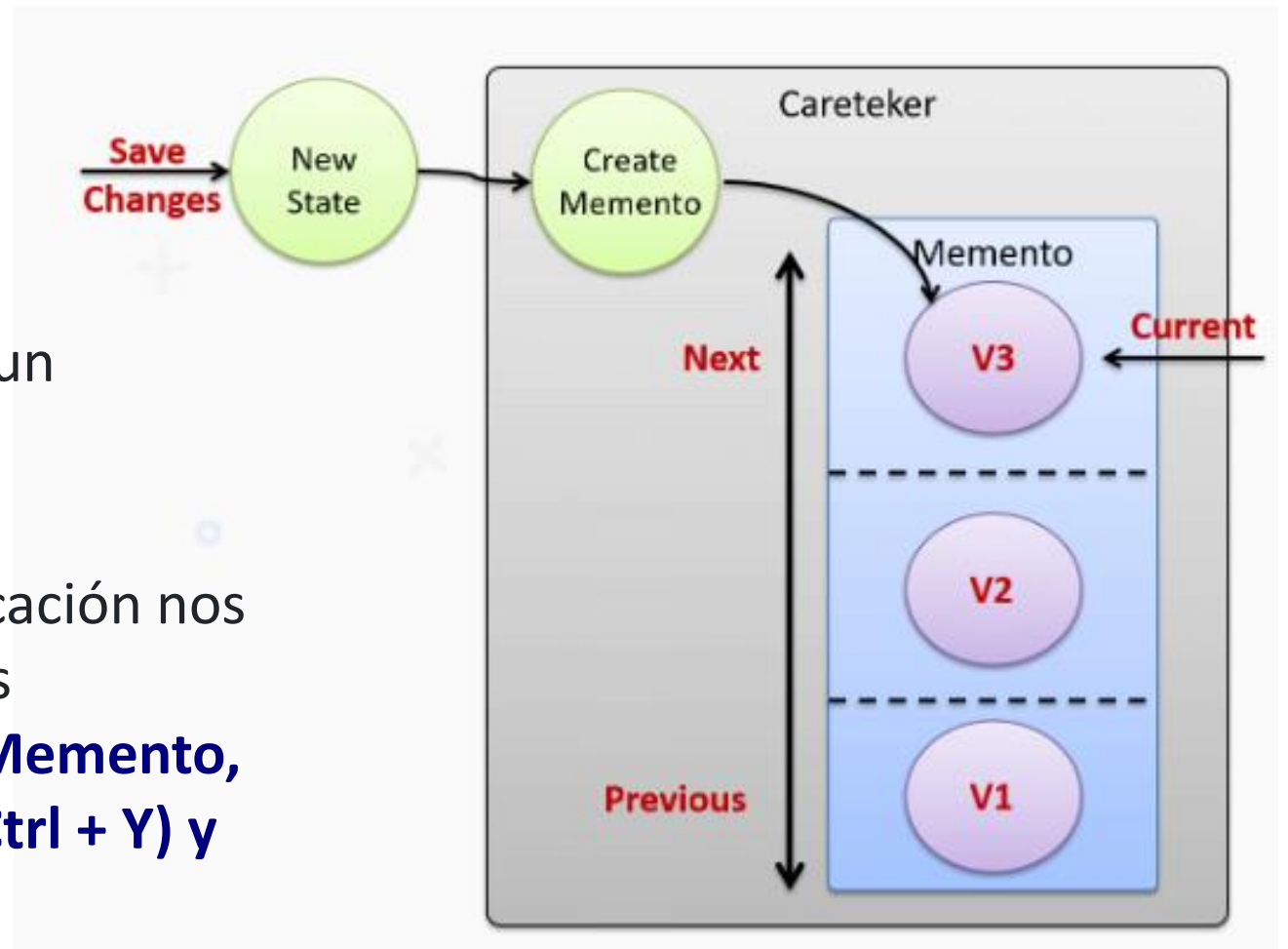
Memento

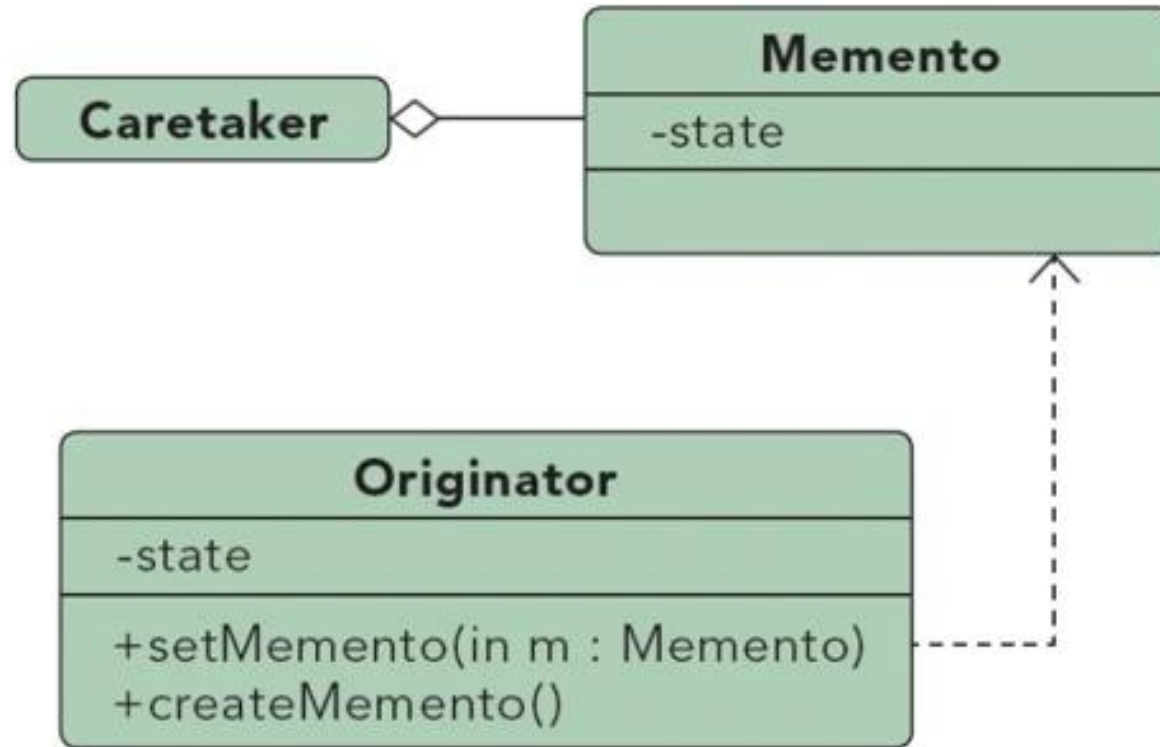
Almacena el estado de un objeto (o del Sistema completo) en un momento dado de manera que se pueda restaurar en ese punto sin revelar detalles de su implementación

Memento

Necesitamos capturar información de un empleado.

Una vez guardemos los datos, la aplicación nos permitirá regresar entre las diferentes versiones almacenadas en el patrón **Memento**, como si fuera una opción de hacer (Ctrl + Y) y deshacer (Ctrl + Z).





Aplicación

Fomenta la encapsulación

57

Utilidad

La captura del estado interno de un objeto, y guardar el estado fuera del objeto para que el objeto puede ser posteriormente restaurado a un estado guardado previamente.



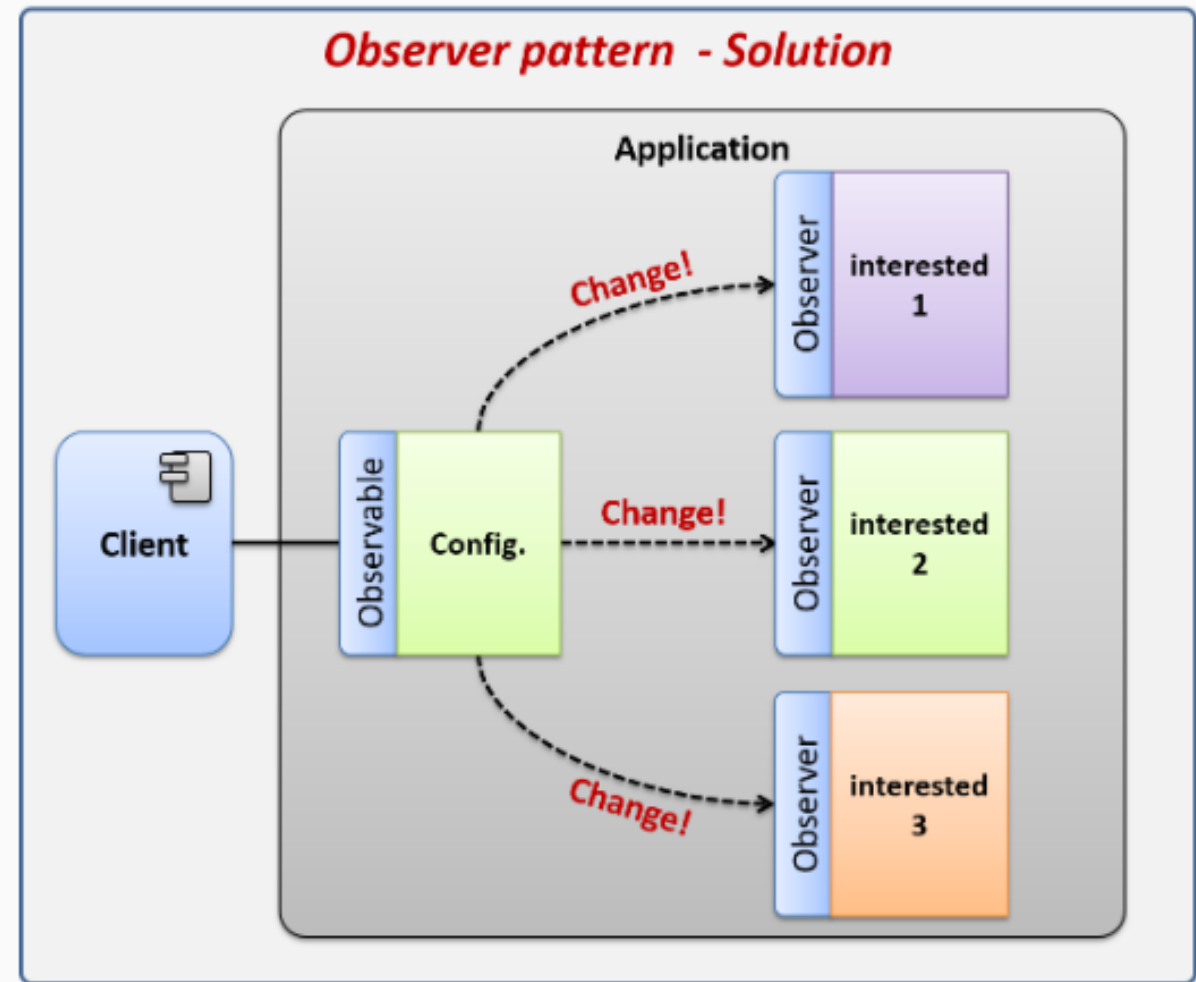
Observer

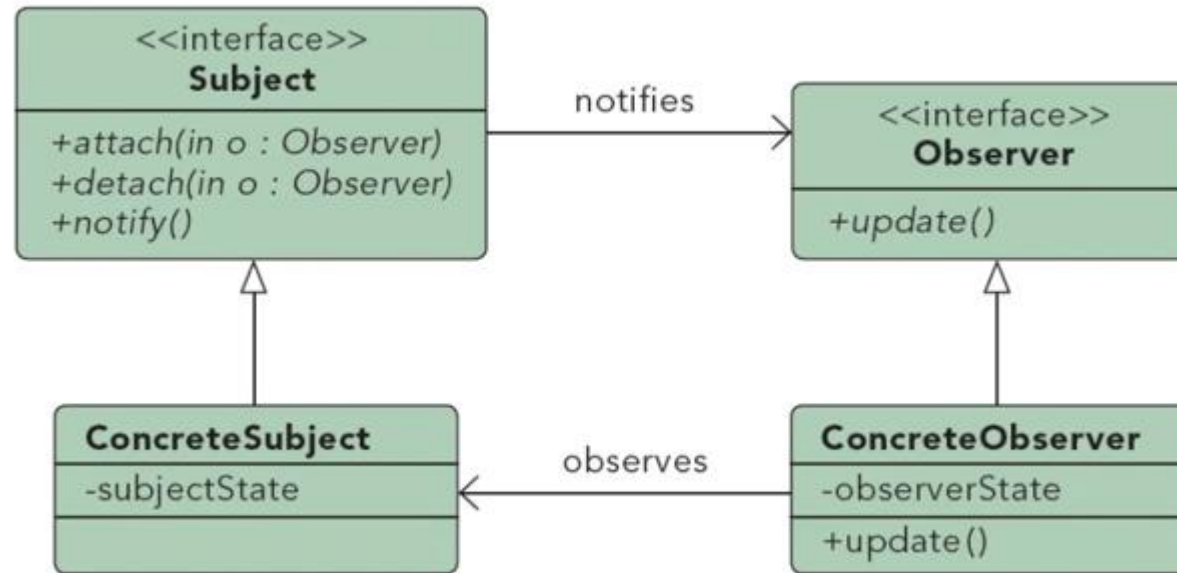
Permite definir un mecanismo de “suscripción” (una dependencia del tipo *uno a muchos* entre objetos), de manera que cuando **uno de los objetos cambia su estado, notifica este cambio** a todos los dependientes.

Observer

Desarrollaremos una aplicación que cargue desde el inicio la configuración del sistema.

Dicha **configuración notificará de forma automática a todos los objetos que estén interesado saber de los cambios**, de modo que no tenga que consultar cada momento si existe algún cambio.





Aplicación

Definir una dependencia 1-n entre objetos de forma que cuando uno cambie de estado todos los Dependientes sean notificados

60

Utilidad

Cuando un objeto debe notificar a otros objetos de diferente naturaleza sin estar acoplados a ellos.

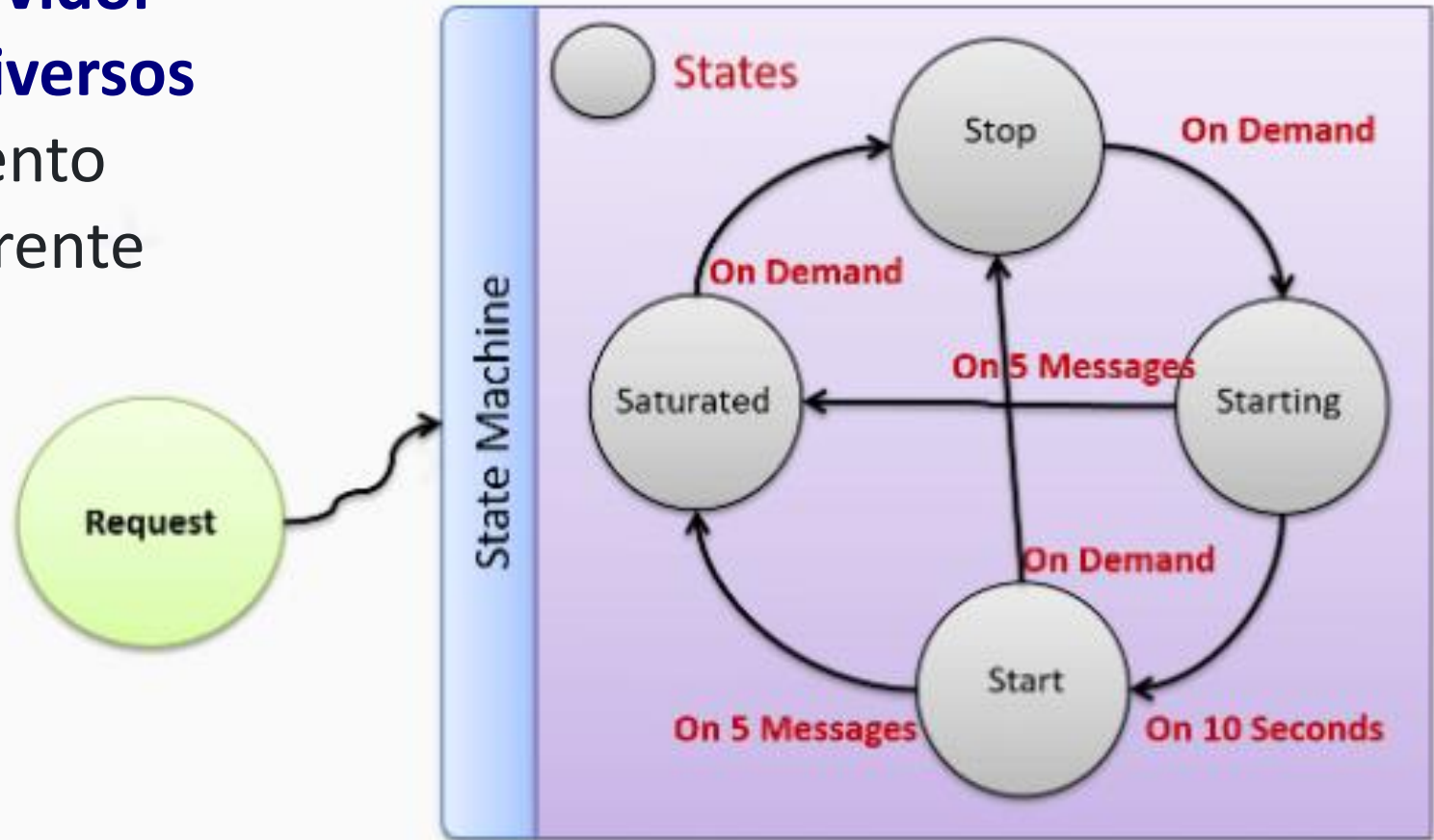
Patrón State

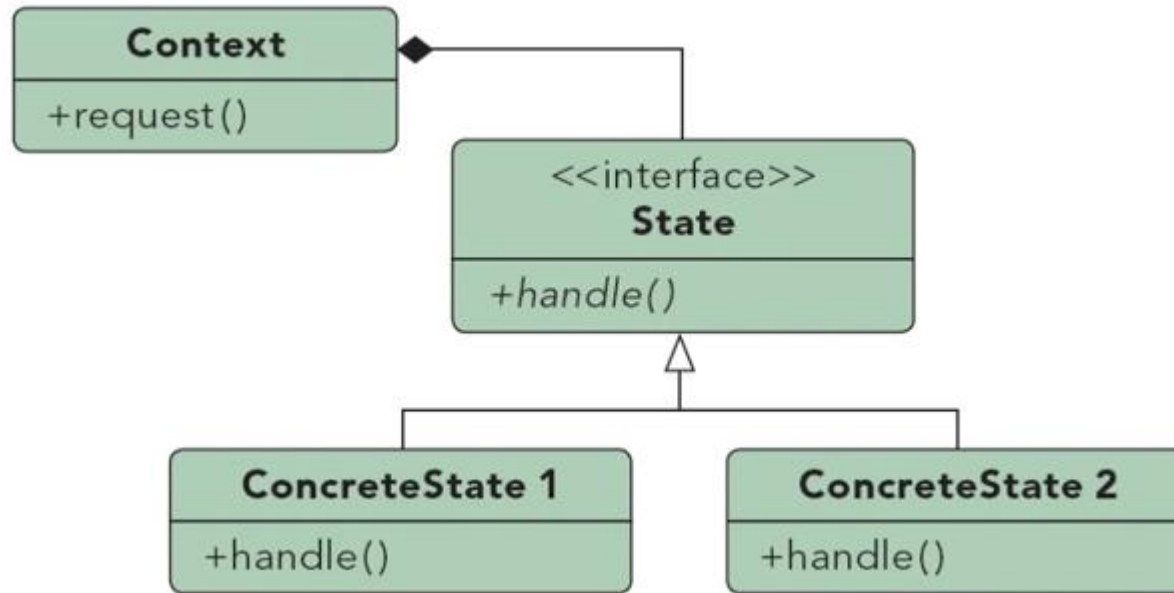
State

Permite a un objeto **alterar su comportamiento** cuando su estado interno cambia.

State

Los estados le indicarán al servidor como comportarse ante los diversos **eventos**, ya que un mismo evento puede tener un resultado diferente según el estado del servidor.





Aplicación

se utiliza cuando el comportamiento de un objeto cambia dependiendo del estado del mismo. Por ejemplo: una alarma puede tener diferentes estados, como desactivada, activada, en configuración. Definimos una interfaz `Estado_Alarma`, y luego definimos los diferentes estados.

Utilidad

El patrón State propone una solución a esta situación, creando básicamente, un objeto por cada estado posible del objeto que lo llama.



Strategy

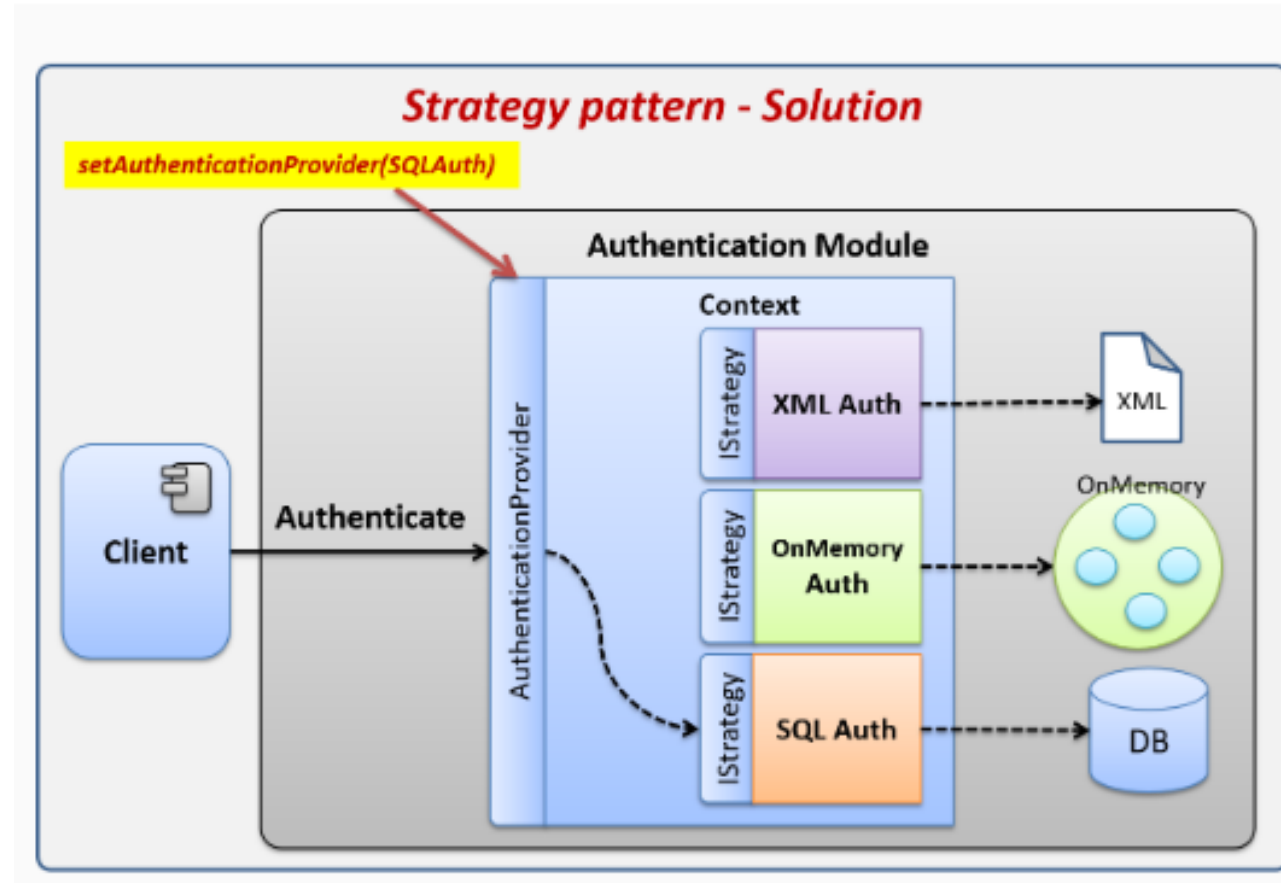
Estas **estrategias** consisten en una familia de algoritmos que están separados del programa real y son autónomos, es decir intercambiables.

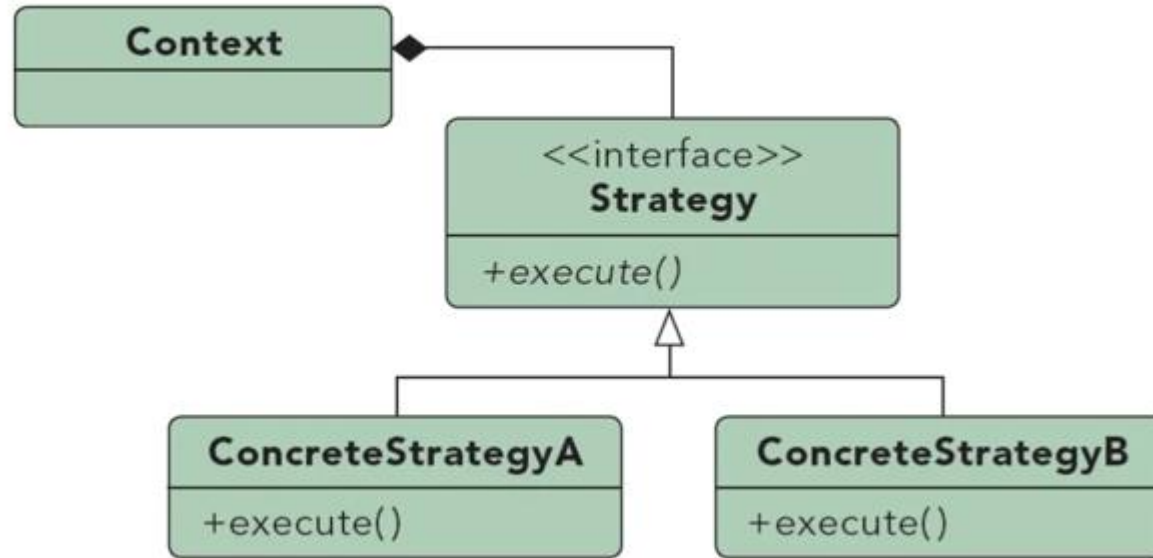
Strategy

El usuario podrá autenticarse mediante una configuración de usuario:

- por XML,
- base de datos o
- una configuración en memoria.

Mediante el patrón Strategy, **nuestro cliente podrá configurar la aplicación para elegir que método de autenticación que le es más conveniente**, todo esto, sin necesidad de programar nada adicional.





Aplicación

Strategy nos permite identificar los aspectos de nuestra aplicación que varían y separarlos de las partes comunes. Favorece la composición sobre la herencia y nos permite programar basándonos en una interfaz o clase abstracta y no en una implementación concreta.

Utilidad

Se requiere múltiples versiones de un algoritmo
El comportamiento de una clase debe ser definido en tiempo de ejecución



Template method

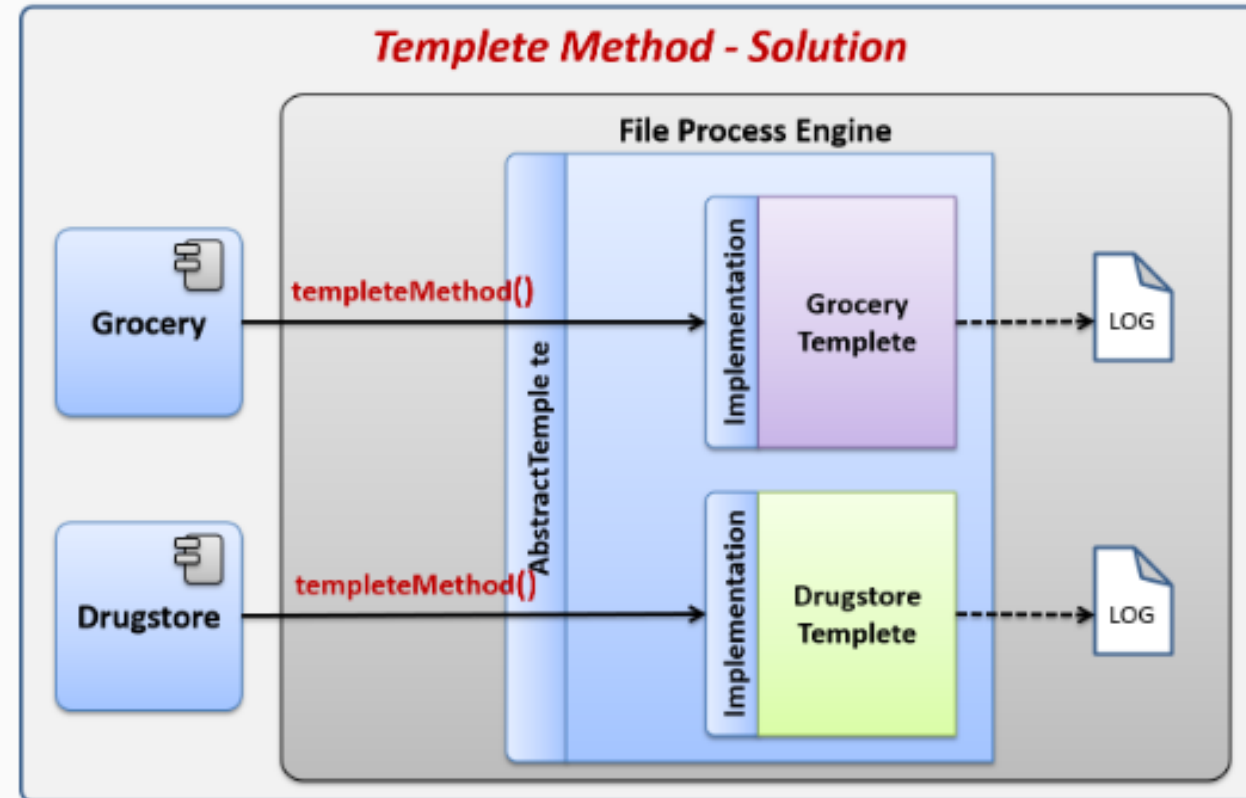
Define una estructura algorítmica, en una super clase, delegando la implementación en las subclases

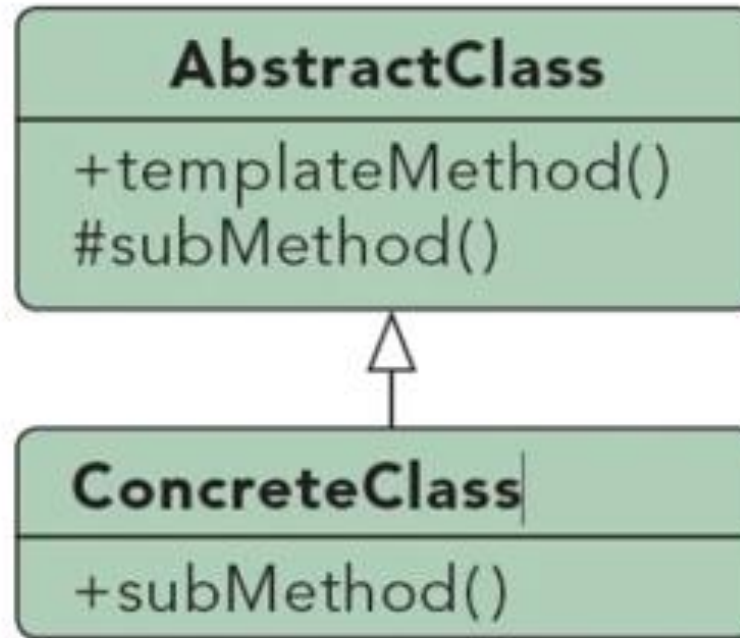
Template method

Desarrollaremos una aplicación que procese archivos de pago, en la cuales las personas pueden pagar sus servicios como la Luz, Agua, TV de paga, Internet, Teléfono, etc.

Las tiendas al final del día generan un archivo de texto plano, el cual es enviado a las compañías correspondientes.

Con el patrón template podemos procesar todo tipo de formato de archivos de una forma simple, clara y genérica.





Aplicación

La principal ventaja es que facilita la reutilización de código, por eso es fundamental este patrón en muchos [Frameworks](#).

69

Utilidad

Muy útiles para construir bibliotecas, pues ayuda a factorizar el comportamiento común de las clases.



Bibliografía

-
- Patrones de diseño, Erich Gamma
 - UML and Patterns de Larman
 - Implementations Patterns Kent Beck
 - <https://refactoring.guru/es/design-patterns>
-